
A GUIDED WALKTHROUGH FOR THE WORKING DEVELOPER

InfraAis

A beginner's manual to a live AIS vessel-tracking service in .NET 8

What this covers: every file in the `InfraAis` project — the background worker, the validator, the repository, the REST API, the tests, and the database schema.

Who it is for: a developer who knows C# the language (classes, methods, loops, collections) but is new to ASP.NET Core — dependency injection, async/await, hosted services, options binding.

How to read it: Part II teaches the seven framework concepts you need. Part III walks the code file by file. Do not skip Part II — Part III assumes it.

Verified against: the working tree as of 11 August 2026, .NET 8.0, all 80 unit tests passing.

Table of Contents

Part I — Orientation

- 1 What this program actually does
 - 2 The architecture at a glance
 - 3 How a .NET project is laid out
-

Part II — The seven concepts you need first

- 4 Nullable reference types, and what `?` means
 - 5 `async`, `await`, and `Task`
 - 6 Dependency injection
 - 7 Service lifetimes, and the scope trap
 - 8 The options pattern
 - 9 Interfaces, and why `IAisRepository` exists
 - 10 `BackgroundService` — code that runs forever
-

Part III — The code, file by file

- 11 `Program.cs` — the composition root
 - 12 `Options/` — typed configuration
 - 13 `Models/AisMessage.cs` — the shape of the feed
 - 14 `Models/` — records and responses
 - 15 `Validation/AisValidator.cs` — the rules
 - 16 `Services/AisIngestionService.cs` — the engine room
 - 17 `Repositories/` — talking to SQL Server
 - 18 `Services/VesselIdentifierResolver.cs`
 - 19 `Controllers/VesselsController.cs` — the API
 - 20 `Utils/NavStatusMapper.cs`
 - 21 The database schema
 - 22 The tests
-

Part IV — Judgment

- 23 Ten things in this code worth questioning
 - 24 Glossary
 - 25 Answers to the checks
-

HOW EACH CHAPTER IS BUILT

Every chapter in Part II and Part III follows the same four beats, deliberately:

1. The concept — the idea in plain language, before any syntax. **2. An analogy or diagram** — something to hang the idea on. **3. The real code** — taken verbatim from your repository, with line numbers, then annotated line by line. **4. Check yourself** — two or three questions. Answers are in Chapter 25. Try them before you look.

Orientation

Before a single line of C#: what the program is for, what shape it has, and how to find your way around a .NET project on disk.

1 • What this program actually does

Every ship above a certain size continuously broadcasts a radio message saying, in effect, *"I am vessel 255805678, I am at 51.4°N 3.1°E, doing 15.6 knots on a heading of 271°."* The system is called **AIS** — Automatic Identification System. It exists so ships do not run into each other, and it is unencrypted, so anyone with a receiver can listen.

Shore stations and satellites pick these broadcasts up and aggregate them. One such aggregator, aisstream.io, republishes the global firehose over a WebSocket. **This program is a client of that firehose.** It connects, listens forever, cleans up what it hears, and files it into a SQL Server database. Then it offers a small REST API so you can ask "where is this ship right now?"

The two kinds of message that matter

AIS defines 27 message types. This program cares about exactly two, and the distinction between them drives most of the design:

Message	Contains	Behaviour
PositionReport	Where the ship is <i>right now</i> — latitude, longitude, speed, course, heading, navigational status.	Arrives constantly — every 2 to 10 seconds for a moving vessel. High volume.
ShipStaticData	Who the ship <i>is</i> — name, IMO number, call sign, ship type, physical dimensions, draught, and the current voyage's destination and ETA.	Arrives roughly every 6 minutes. Low volume.

So the same vessel produces a steady stream of "where I am" and an occasional "who I am". The database mirrors that split exactly: a [vessels](#) table (one row per ship, updated in place) and a [positions](#) table (one row per report, append-only, growing forever).

THE TWO IDENTIFIERS — MMSI AND IMO

Ships have two numbers and they are not interchangeable. This trips people up constantly, so learn it now.

MMSI (Maritime Mobile Service Identity) is 9 digits. It identifies the *radio installation*, not the hull. It is present in every single message. But it changes when a ship changes flag state — sell a ship from Germany to Panama and it gets a new MMSI.

IMO number is 7 digits. It is welded to the *hull* and never changes for the life of the ship, across every rename and reflagging. But it only appears in static messages, never in position reports.

So MMSI is *always available but impermanent*; IMO is *permanent but rarely broadcast*. That is why `mmsi` is the primary key of the `vessels` table, why `imo` is nullable, and why an entire class (`VesselIdentifierResolver`, Chapter 18) exists just to translate one into the other.

What "cleaning up" means

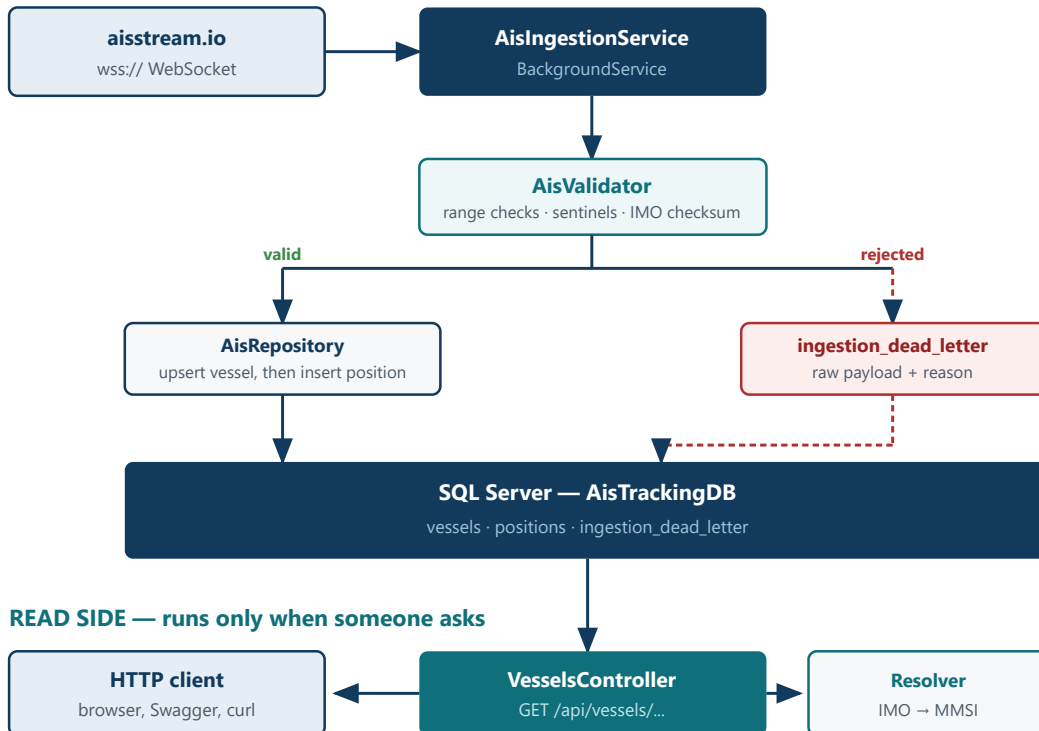
Raw AIS is dirty. Transponders are misconfigured, radio bursts collide, and the standard itself uses sentinel values — magic numbers meaning "I don't know" — rather than a proper null. Speed over ground arrives as `102.3` when unknown. Course arrives as `360`. Heading arrives as `511`. Latitude arrives as `91`, which is not a real latitude at all.

If you stored those literally you would later compute an average fleet speed and get a nonsense answer, because a hundred ships "did" 102.3 knots. So a large part of this program — the whole of `AisValidator` — is about turning "I don't know" into a genuine database `NULL`, and about rejecting messages that are simply not usable.

2 · The architecture at a glance

One process does two unrelated jobs at the same time. Hold this picture in your head; the rest of the manual is just detail hung on it.

WRITE SIDE — runs forever, nobody asks it to



The whole system. The database is the only thing the two halves share.

Notice what is *not* in that picture: there is no queue, no message bus, no cache. The write side and the read side never call each other. They communicate only through the database, and they do not even know the other exists. That is a deliberate and rather clean piece of design — and it is why the read side can be tested with no feed running, and the write side works fine with no API traffic.

ANALOGY — THE HARBOUR MASTER'S OFFICE

Picture a harbour master's office. In the back room a clerk sits at the radio all day, every day, writing down every ship report he hears into a ledger. He never stops, nobody tells him to start, and he does not care whether anybody ever reads the ledger.

At the front counter, a second clerk waits. When a visitor comes in and asks "where is the *Buxcliff* right now?", she walks to the ledger, looks it up, and answers. If no visitor comes all day, she does nothing.

The back-room clerk is **AisIngestionService**. The front-counter clerk is **VesselsController**. The ledger is SQL Server. They never speak to each other — and this is precisely the shape of the program.

Why one process and not two?

A fair question, and one an interviewer will ask. Nothing forces the worker and the API to share a process; they could be two deployables. They are together here because it is Phase 1 of a project — one thing to run, one thing to configure, one thing to debug.

The cost is real, though: they now scale together and fail together. If you wanted three API instances behind a load balancer you would get three ingestion workers too, all writing the same rows three times over. When that day comes, the split is easy precisely *because* they only share a database. Keep it in mind as a known trade-off rather than an oversight.

CHECK YOURSELF

1. A ship is sold and reflagged from Germany to Panama. Which of its two identifiers changes, and which table in this database would end up with a stale or duplicated row as a result?
2. Why can the position report not simply carry the IMO number, saving all this resolving work?
3. If you deployed this application to three servers behind a load balancer without changing anything, what specifically would go wrong?

3 · How a .NET project is laid out

If your C# so far has been console applications, the folder structure of a real service can look arbitrary. It is not. Here is the whole repository, and what each piece is for.

```
infra-AIS/
├── InfraAis/                                ← the application itself
│   ├── Controllers/                        HTTP endpoints (the read side)
│   │   └── VesselsController.cs
│   ├── Models/                            plain data-holding classes
│   │   ├── AisMessage.cs                 shape of what the FEED sends us
│   │   ├── VesselRecord.cs               shape of what we STORE
│   │   ├── PositionRecord.cs
│   │   ├── LatestPositionRecord.cs
│   │   ├── LatestPositionResponse.cs     shape of what the API RETURNS
│   │   ├── VesselInfoResponse.cs
│   │   └── VesselIdentifier.cs
│   ├── Options/                          strongly-typed configuration
│   │   ├── AisStreamOptions.cs
│   │   ├── IngestionOptions.cs
│   │   └── AisDbOptions.cs
│   ├── Repositories/                     all SQL lives here and nowhere else
│   │   ├── IAisRepository.cs
│   │   └── AisRepository.cs
│   ├── Services/                         business logic and long-running work
│   │   ├── AisIngestionService.cs
│   │   └── VesselIdentifierResolver.cs
│   ├── Utils/
│   │   └── NavStatusMapper.cs
│   ├── Validation/
│   │   └── AisValidator.cs
│   ├── Program.cs                        startup - wires everything together
│   ├── appsettings.json                 non-secret configuration
│   └── InfraAis.csproj                   the project definition
├── InfraAis.Tests/                       ← a SEPARATE project, for tests only
├── schema.aistrackingdb.sql
└── README.md
```

Two things about this are worth stating explicitly, because they are conventions rather than rules the compiler enforces.

The folder names are architectural claims. A file in `Repositories/` is promising that it talks to the database and does nothing clever. A file in `Validation/` is promising it is pure logic with no I/O. When someone puts a SQL query inside a controller, nothing breaks — but the promise is broken, and the next person can no longer trust the map. Keep the promises.

Tests are a different project. `InfraAis.Tests` compiles to its own DLL and references `InfraAis`. This is why tests can see your classes but your application can never accidentally ship test code. The reference goes one way only.

The project file

```
InfraAis/InfraAis.csproj
```

```

<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net8.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <UserSecretsId>fef3a423-c6e2-4981-b93c-4edbb5832e0e</UserSecretsId>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.Data.SqlClient" Version="7.0.2" />
    <PackageReference Include="Swashbuckle.AspNetCore" Version="6.6.2" />
  </ItemGroup>

</Project>

```

This tiny file explains several mysteries you would otherwise hit in Part III.

`Sdk="Microsoft.NET.Sdk.Web"`

The *web* SDK, not the plain one. This is what gives you `WebApplication.CreateBuilder`, the built-in Kestrel web server, and automatic loading of `appsettings.json`. A console app would say `Microsoft.NET.Sdk` and none of that would exist.

`<Nullable>enable</Nullable>`

Turns on nullable reference types across the whole project. This single line is why you see `string?` and `VesselRecord?` everywhere. Chapter 4 is devoted to it.

`<ImplicitUsings>enable</ImplicitUsings>`

The compiler silently adds about fifteen `using` directives to every file — `System`, `System.Linq`, `System.Threading.Tasks`, `Microsoft.AspNetCore.Builder` and friends. **This is why `Program.cs` can use `WebApplication` and `ILogger` without importing anything.** If you have ever wondered why a file compiles despite obviously missing an import, this is the answer.

`<UserSecretsId>`

A per-machine identifier for the secrets store. Your `aisstream.io` API key is *not* in `appsettings.json` — it lives outside the repository in your user profile, keyed by this GUID, so it can never be committed by accident. Chapter 12 covers how it is loaded.

`PackageReference`

Exactly two external dependencies. `Microsoft.Data.SqlClient` is the SQL Server driver; `Swashbuckle` generates the Swagger UI. Notice what is *absent*: no Entity Framework, no Dapper, no AutoMapper, no MediatR. This project writes its own SQL and does its own mapping by hand.

CONCEPT — NAMESPACES VERSUS FOLDERS

C# does not require a file's namespace to match its folder. Convention says it should, and this project follows it: `Repositories/AisRepository.cs` declares `namespace InfraAis.Repositories;`

Note the semicolon rather than braces. That is a *file-scoped namespace*, a C# 10 feature: it applies to the whole file and saves you one level of indentation. Every file in this project uses it.

CHECK YOURSELF

1. `Program.cs` uses the type `WebApplication` but has no `using Microsoft.AspNetCore.Builder;` at the top. Why does it compile?
2. Where does the AIS API key physically live on disk, and why not in `appsettings.json` ?
3. The test project references the application project. What would go wrong if you made the application reference the test project as well?

PART TWO

The seven concepts you need first

These seven ideas account for nearly everything in this codebase that is not plain C#. Learn them here, in isolation, and Part III becomes a walk rather than a climb.

4 · Nullable reference types, and what `?` means

The concept

For twenty years, every reference type in C# could be `null`, and the compiler never warned you. The result was the single most common crash in .NET: `NullReferenceException`. Tony Hoare, who invented the null reference in 1965, later called it his "billion-dollar mistake".

C# 8 introduced a fix that costs nothing at runtime. When you write `<Nullable>enable</Nullable>` in the project file — as this project does — the meaning of type declarations changes:

Declaration	Means	Compiler behaviour
<code>string Name</code>	"This is never null."	Warns if you assign null, or leave it unassigned in a constructor.
<code>string? Name</code>	"This might be null."	Warns if you use it without checking first.

THE CRUCIAL CAVEAT

These are **warnings, not errors**, and they are erased at compile time. A `string` without a question mark can still hold null at runtime — if the value came from JSON deserialization, from a database, or from code compiled without the feature. The compiler is giving you a very good static hint, not a runtime guarantee. Treat it as a seatbelt, not a force field.

ANALOGY — THE PARCEL AND THE LABEL

`string Name` is a parcel labelled "**guaranteed to contain something**". You may open it without looking. `string? Name` is labelled "**may be empty**" — the compiler will nag you every time you reach inside without first checking that something is there.

But the labels are applied by the sender, not verified by an inspector. A mislabelled parcel still explodes when you open it. That is the difference between a warning and a guarantee.

Where you meet it in this project

Open `Models/VesselRecord.cs`. Every question mark there is a factual statement about ships:

InfraAis/Models/VesselRecord.cs

```
5    public long Mmsi { get; set; }
6    public int? Imo { get; set; }
7    public string? Name { get; set; }
8    public string? CallSign { get; set; }
9    public int? ShipType { get; set; }
10   public DateTime TimestampUtc { get; set; }
```

line 5 – long Mmsi

No question mark. Every AIS message carries an MMSI; without one we could not file the row at all. The type is asserting a domain fact.

line 6 – int? Imo

Question mark. The IMO arrives only in static messages, and even then it is discarded if its checksum fails. Most vessel rows will have `NULL` here for a long time. The `?` forces every consumer to handle that.

line 7 – string? Name

Same reasoning. A vessel first seen through a position report has no name until a static message arrives minutes later.

Read that class again and notice what it is really doing: **it is documenting the AIS protocol in the type system**. Anyone reading it learns which facts about a ship are always available and which are not, without opening a specification. That is nullable reference types being used well.

The four operators you will see

These appear throughout Part III. Learn them now and the code stops looking cryptic.

```
// 1. ?. – null-conditional. "If the left side is null, stop and yield null."
DimToBow = (short?)sd.Dimension?.A,    // AisIngestionService line 241
//   If Dimension is null, the whole expression is null. No crash.

// 2. ?? – null-coalescing. "Use the left, unless it is null, then use the right."
.Value = (object?)vessel.Imo ?? DBNull.Value;    // AisRepository line 54
//   C# null and SQL NULL are different things; this translates one to the other.

// 3. is null / is not null – the preferred null test.
if (pr is null) return;    // AisIngestionService line 151
//   Safer than == because it cannot be hijacked by an overloaded operator.

// 4. ! – the null-forgiving operator. "Trust me, compiler."
Assert.IsType<NotFoundObjectResult>(result.Result).Value!.ToString();
//   Silences the warning. Proves nothing. Used in tests where a failure is fine.
```

ON THE `!` OPERATOR

In this codebase `!` appears only in test files, which is exactly right. In test code an unexpected null should crash the test — that is the failure signal. In production code, reach for `!` only when you can articulate why null is impossible. "It shuts the warning up" is not a reason; it is how you convert a compile-time warning into a 3 a.m. incident.

CHECK YOURSELF

1. `sd.Dimension?.A` — if `Dimension` is null, what is the value of the whole expression, and what type must the variable receiving it be?
2. Why is `Mmsi` declared `long` and not `long?`, when `Imo` is `int`? Answer in terms of ships, not C#.
3. A colleague adds `!` to production code to clear a build warning. Give the one-sentence argument for why that is worse than leaving the warning.

5 • `async`, `await`, and `Task`

The concept

This program spends almost all of its life waiting. Waiting for the next message to arrive over the network. Waiting for SQL Server to acknowledge an insert. Waiting for thirty seconds to elapse before printing a summary.

Waiting is not the problem. The question is **what the thread does while waiting**.

A thread is an expensive operating-system resource — roughly a megabyte of stack each, and the OS must schedule every one of them. The old, synchronous way blocks the thread: it sits there, consuming memory, doing nothing, until the network replies. Serve a thousand concurrent requests that way and you need a thousand blocked threads, and the machine falls over.

`async` / `await` solves this. At an `await`, if the operation has not finished yet, **the method returns control to the caller and the thread is released to do other work**. When the result eventually arrives, the runtime finds a thread — possibly a different one — and resumes the method from exactly where it left off, with all its local variables intact.

ANALOGY — THE RESTAURANT

A **synchronous** waiter takes your order to the kitchen and then stands at the pass, staring at the chef, until your food is ready. He is fully occupied and no other table gets served. To serve twenty tables you must hire twenty waiters.

An **asynchronous** waiter takes your order, hands it to the kitchen, and immediately goes to serve table four. When your food appears in the hatch, whichever waiter is free carries it over. One waiter serves twenty tables — because cooking, not carrying, was the slow part.

`await` is the moment the waiter hands the order to the kitchen and walks away. Crucially, **it is not "do this on another thread"** — nobody hired a second waiter. It is "release me until this is done".

Reading the three keywords

```
// From AisRepository.InsertPositionAsync
public async Task InsertPositionAsync(PositionRecord p)
{
    await using var conn = new SqlConnection(_connectionString);
    await conn.OpenAsync();
    ...
    await cmd.ExecuteNonQueryAsync();
}
```

Task

The return type. Read it as **"a receipt for work that will finish later"**. `Task` alone means "will finish, no value". `Task<long?>` means "will finish and hand back a `long?`". It is the exact analogue of a JavaScript `Promise`.

async

A marker on the method telling the compiler to rewrite it into a state machine so it can be suspended and resumed. **It changes nothing about how the method is called.** It is a note to the compiler, not to the caller — which is why `async` never appears in an interface signature. Look at `IAisRepository`: the methods return `Task`, but not one of them says `async`.

await

The suspension point. "Unwrap this Task. If it is not finished, release the thread and resume me here when it is." Every `await` is a place your method may pause for an unbounded time.

await using

`using` means "dispose this when the block ends". `await using` means "dispose it *asynchronously*" — for a database connection, returning it to the pool may itself involve network chatter, and this avoids blocking a thread to do so.

The naming convention

Every asynchronous method in this codebase ends in `Async`: `InsertPositionAsync`, `GetLatestPositionAsync`, `ResolveToMmsiAsync`. This is a .NET-wide convention with a practical purpose — the suffix is a visible warning at the call site that you must `await` the result. Follow it in any code you add.

THE MISTAKE THAT WILL BITE YOU: FORGETTING `AWAIT`

Consider:

```
repo.InsertPositionAsync(record);           // no await - compiles fine!  
await repo.InsertPositionAsync(record);     // correct
```

The first line compiles and produces only a mild warning. It starts the insert and immediately moves on without waiting. Two consequences follow, both nasty: you have no idea whether the insert succeeded, and if it throws, the exception is **swallowed silently** — it goes into a Task nobody ever looks at. You get missing rows and an empty log.

This project does exactly that on purpose in one place, at `AisIngestionService` line 43. Chapter 16 examines whether it was the right call.

CHECK YOURSELF

1. Does `await` create a new thread? Explain your answer using the restaurant.
2. `IAisRepository` declares `Task<VesselRecord?> GetVesselAsync(long mmsi);` with no `async` keyword, yet `AisRepository` implements it with `async`. Why is that legal?
3. You delete the word `await` from `await cmd.ExecuteNonQuery();`. The build still succeeds. Name the two distinct things that can now go wrong at runtime.

6 · Dependency injection

The concept

A class needs other objects to do its job. `VesselsController` needs a repository, a resolver, and a logger. There are two ways it can get them.

The direct way — build them itself:

```
// NOT how this project works
public class VesselsController
{
    private readonly AisRepository _repo = new AisRepository(???);
    // ...and now the controller must also know the connection string,
    // how to read configuration, and how to build every dependency of
    // the repository, and of ITS dependencies, forever.
}
```

The injected way — demand them and let someone else supply them:

InfraAis/Controllers/VesselsController.cs — lines 17-25

```
public VesselsController(
    IVesselIdentifierResolver resolver,
    IAisRepository repo,
    ILogger<VesselsController> logger)
{
    _resolver = resolver;
    _repo = repo;
    _logger = logger;
}
```

The controller now states its needs and receives them ready-made. It does not know that `IAisRepository` is really `AisRepository`, does not know there is a connection string in the world, and cannot be broken by changes to how repositories are constructed.

Search the entire project for `new VesselsController()` — outside the test file, it does not exist. Nothing constructs it. **The framework does**, using the DI container.

ANALOGY — THE OPERATING THEATRE

A surgeon does not wander the hospital fetching her own scalpel, sutures, and anaesthetist. She states what the operation requires; a theatre nurse lays it all out before she scrubs in. The surgeon can then concentrate entirely on surgery.

The DI container is the theatre nurse. Your constructor parameter list is the instrument tray. And note the consequence: to rehearse the operation you simply hand over practice instruments — which is exactly what the tests do with mocks in Chapter 22.

How the container knows what to supply

Because `Program.cs` told it. Registration is the whole of the container's knowledge:

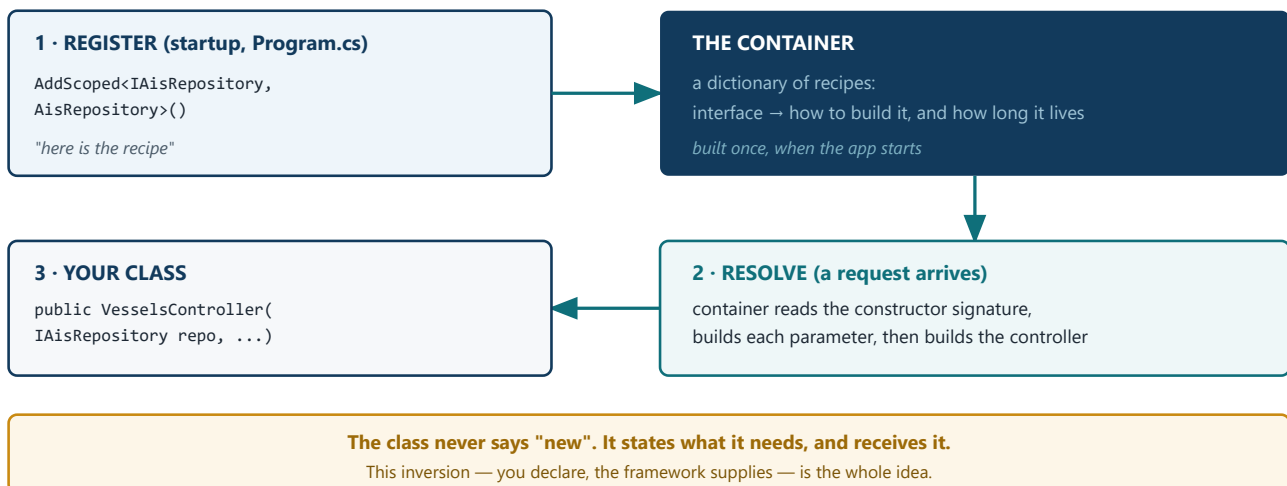
InfraAis/Program.cs — lines 17-18

```
builder.Services.AddScoped<IAisRepository, AisRepository>();  
builder.Services.AddScoped<IVesselIdentifierResolver, VesselIdentifierResolver>();
```

Read each as a sentence: **"when somebody asks for the interface on the left, build them the concrete class on the right."** Two things follow from this and both matter.

First, the wiring lives in exactly one file. To learn what this application is made of, you read `Program.cs` and nothing else. That file is conventionally called the **composition root**.

Second, swapping an implementation is a one-line change. Point `IAisRepository` at a `PostgresAisRepository` and every consumer keeps working, unmodified, because none of them ever named the concrete type.



THE ERROR YOU WILL CERTAINLY HIT

Unable to resolve service for type 'InfraAis.Repositories.IAisRepository' while attempting to activate 'InfraAis.Controllers.VesselsController'.

Translation: **you added a constructor parameter but forgot to register it in `Program.cs`**. The container was asked for something it has no recipe for. Note that this failure happens at *runtime*, on the first request — not at compile time. That is the standing cost of dependency injection, and it is why Chapter 23 argues for start-up validation.

CHECK YOURSELF

1. You add a parameter `IClock clock` to `VesselsController`'s constructor and nothing else. The project compiles. When and how does it fail?
2. Why does `AddScoped` take two type arguments rather than one? What does each mean?
3. The tests construct `new VesselsController(...)` directly with mocks and no container at all. What property of constructor injection makes that possible?

7 · Service lifetimes, and the scope trap

The concept

When the container builds an object, how long should it keep it? Three answers exist, and choosing wrongly causes some of the nastiest bugs in ASP.NET Core.

Lifetime	One instance per...	Use for
Singleton	...application. One object, ever, shared by everybody.	Expensive, stateless, thread-safe things: caches, <code>HttpClient</code> factories, hosted services.
Scoped	...HTTP request. A new object per request, shared within it.	Anything holding per-request state: database connections, units of work, repositories.
Transient	...injection. A brand-new object every single time it is asked for.	Cheap, stateless helpers.

This project uses two of them:

```
builder.Services.AddHostedService<AisIngestionService>();           // effectively a SINGLETON
builder.Services.AddScoped<IAisRepository, AisRepository>();       // SCOPED
builder.Services.AddScoped<IVesselIdentifierResolver, ...>();       // SCOPED
```

And now the trap

Here is a genuinely important problem, and this codebase solves it correctly — but the solution looks so odd that it is worth understanding before you meet it in Chapter 16.

A "scope" in ASP.NET Core normally means **one HTTP request**. The framework opens a scope when a request arrives and disposes it when the response is sent. That is how scoped services get their lifetime.

But the ingestion worker is not an HTTP request. It is a singleton that starts with the application and runs for weeks. There is no request, so there is no scope, so it cannot be handed a scoped repository at all.

WHY YOU CANNOT SIMPLY INJECT IT ANYWAY

Suppose you did put `IAisRepository` directly in the worker's constructor. Two things happen, both bad.

It fails immediately. ASP.NET Core validates this at start-up and throws: *"Cannot consume scoped service 'IAisRepository' from singleton 'AisIngestionService'."*

And if you defeated that check, it would be worse. The single repository instance would be captured by the singleton and live for the entire application lifetime. This is called a **captive dependency**: an object with a short intended life, imprisoned inside a long-lived one. Any per-operation state it holds leaks across every message for weeks.

The fix is to inject not the service but a **factory for scopes**, then open and close a scope manually around each unit of work:

```
using var scope = _scopeFactory.CreateScope();
var repo = scope.ServiceProvider.GetRequiredService<IAisRepository>();
```

`_scopeFactory.CreateScope()`

Manually opens a scope — the same kind of scope an incoming HTTP request would have opened. It is now a legitimate place for scoped services to live.

`using var`

Disposes the scope when the enclosing method returns. Everything scoped that was created inside it is disposed too. Miss this `using` and you leak a scope per message — thousands per minute.

`GetRequiredService<IAisRepository>()`

Asks the scope for the repository. "Required" means it throws a clear exception if unregistered, rather than returning null and failing mysteriously later. Prefer it over `GetService`.

ANALOGY — THE NIGHT PORTER

Day staff each get a locker for their shift and hand back the key at the end — that is **scoped**.

The night porter works continuously and was never issued a shift locker. If he took one on his first night and kept it for a year, everything he ever stored would pile up inside — that is the captive dependency.

What he actually does is borrow a locker for each task and return it immediately. `CreateScope()` is borrowing the locker; the `using` is handing the key back.

CHECK YOURSELF

1. What exactly is a "scope" during a normal web request, and who opens and closes it?
2. Give the specific error message you would get from injecting `IAisRepository` straight into `AisIngestionService`.
3. Someone deletes the word `using` from `using var scope = ...`. The code still compiles and runs. Describe what degrades, and over what timescale.

8 · The options pattern

The concept

Configuration arrives as text: JSON files, environment variables, command-line arguments. Reading it directly by string key is miserable and fragile:

```
// The bad way - three separate ways to fail at runtime
var delay = int.Parse(Configuration["Ingestion:ReconnectBaseDelaySeconds"]);
//           ^ typo the key → null → Parse throws
//           ^ no default if missing
//           ^ no compiler help, no IntelliSense
```

The options pattern replaces the string keys with a class. You define the shape you want, the framework fills it in from configuration, and from then on it is an ordinary strongly-typed object.

The four steps, followed through the project

Step 1 — describe the shape. A plain class with properties and sensible defaults:

InfraAis/Options/IngestionOptions.cs

```
public class IngestionOptions
{
    public const string SectionName = "Ingestion";

    public int FutureSkewMinutes { get; set; } = 5;
    public int ReconnectBaseDelaySeconds { get; set; } = 1;
    public int ReconnectMaxDelaySeconds { get; set; } = 30;
    public int SummaryIntervalSeconds { get; set; } = 30;
}
```

The `const string SectionName` is a small but excellent habit: the magic string that ties this class to the JSON lives *on the class it describes*, not scattered across `Program.cs`.

Step 2 — the JSON provides values. Property names must match the keys:

```
"Ingestion": {
  "FutureSkewMinutes": 5,
  "ReconnectBaseDelaySeconds": 1,
  "ReconnectMaxDelaySeconds": 30,
  "SummaryIntervalSeconds": 30
}
```

Step 3 — bind them together in `Program.cs`:

```
builder.Services
    .AddOptions<IngestionOptions>()
    .Bind(builder.Configuration.GetSection(IngestionOptions.SectionName));
```

`Bind` uses reflection to match each JSON key to a property of the same name. Anything absent keeps the default from step 1 — so deleting the whole `"Ingestion"` section leaves the application running on its defaults rather than crashing.

Step 4 — consume it in a constructor:

```
public AisIngestionService(..., IOptions<IngestionOptions> ingestion, ...)  
{  
    _ingestion = ingestion.Value;  
}
```

WHY IOPTIONS<T> AND NOT JUST T ?

`IOptions<T>` is a **wrapper** with a single member, `.Value`. The indirection exists so the framework can defer building the settings object, and so it can offer variants with different refresh behaviour:

Interface	Lifetime	Reads configuration...
<code>IOptions<T></code>	Singleton	Once, at first access. Frozen for the life of the app.
<code>IOptionsSnapshot<T></code>	Scoped	Once per request — picks up file edits between requests.
<code>IOptionsMonitor<T></code>	Singleton	Every access, with change callbacks. Survives in long-lived services.

This project uses plain `IOptions<T>` everywhere. The consequence is concrete and worth knowing: **editing `appsettings.json` while the service is running changes nothing**. The values were captured at start-up. For reconnect parameters that is arguably the right choice — you do not want them mutating mid-flight — but it is a decision, and it is invisible in the code.

Where the API key really comes from

Configuration is layered. Later sources override earlier ones, and the default order is:

```
appsettings.json  
  ↓ overridden by  
appsettings.{Environment}.json    // e.g. appsettings.Development.json  
  ↓ overridden by  
User Secrets                       // development only - outside the repo  
  ↓ overridden by  
Environment variables              // AisStream__ApiKey (double underscore = colon)  
  ↓ overridden by  
Command-line arguments
```

This is why `appsettings.json` can safely ship `"ApiKey": ""`. The real key is set once with:

```
dotnet user-secrets set "AisStream:ApiKey" "your-key" --project InfraAis
```

and stored in your user profile, outside the repository, where Git can never see it. In production the same value arrives as the environment variable `AisStream__ApiKey` — note the double underscore, which is how

you express a nested section where a colon is not legal in a variable name.

CHECK YOURSELF

1. You delete the entire `"Ingestion"` block from `appsettings.json`. Does the app crash? What is `SummaryIntervalSeconds` at runtime?
2. You edit `appsettings.json` while the service is running, changing the summary interval to 5. What happens, and why?
3. An environment variable `AisStream__ApiKey` and a user secret `AisStream:ApiKey` are both set to different values. Which wins?

9 · Interfaces, and why `IAisRepository` exists

The concept

An interface is a contract with no implementation — a list of what a type can do, with no statement of how.

InfraAis/Repositories/IAisRepository.cs

```
public interface IAisRepository
{
    Task UpsertVesselAsync(VesselRecord vessel);
    Task InsertPositionAsync(PositionRecord position);
    Task InsertDeadLetterAsync(string rawPayload, string reason);
    Task<long?> GetMmsiByImoAsync(int imo);
    Task<LatestPositionRecord?> GetLatestPositionAsync(long mmsi);
    Task<VesselRecord?> GetVesselAsync(long mmsi);
}
```

Six methods, no bodies, no SQL, no connection string. Read as English it says: *anything calling itself an AIS repository must be able to do these six things*. `AisRepository` is one way to satisfy that, using SQL Server. A test double is another.

A beginner's fair objection: there is only one real implementation, so why bother? Three reasons, and the third is the one that pays.

Reason 1 — it names the seam

The interface is where "business logic" stops and "database" begins. That boundary is now a file you can point at, and any pull request adding SQL somewhere else is visibly wrong.

Reason 2 — it allows substitution

Moving to PostgreSQL means writing a second implementation and changing one line of `Program.cs`. No consumer changes, because no consumer ever named `AisRepository`.

Reason 3 — it makes the code testable, and this is the real payoff

Without the interface, testing the controller would require a running SQL Server with known data in it — slow, flaky, and impossible on a build agent. With it:

InfraAis.Tests/VesselsControllerTests.cs — lines 16-20

```
var resolver = new Mock<IVesselIdentifierResolver>();
var repo = new Mock<IAisRepository>();
var ctrl = new VesselsController(resolver.Object, repo.Object,
    new Mock<ILogger<VesselsController>>().Object);
```

`Mock<T>` manufactures a fake implementation on the fly. You then dictate exactly what it returns:

```
repo.Setup(r => r.GetLatestPositionAsync(255805678))
    .ReturnsAsync((LatestPositionRecord?)null);
```

"When anyone asks for the latest position of that MMSI, return null." Now you can test the "vessel exists but has no positions" branch **deterministically, in 31 milliseconds, with no database at all** — and, more importantly, you can test it for a situation you would struggle to create in a real database on demand.

ANALOGY — THE WALL SOCKET

A three-pin socket is an interface. Your kettle does not know or care whether the electricity behind it comes from a coal station, a wind farm, or a generator in the garden. It knows the contract: three pins, 230 volts.

Change the supply and the kettle is unaffected. Test the kettle on a bench supply and it works identically. That indifference is the entire value — and it exists only because the plug is a contract rather than a direct weld to the power station.

CHECK YOURSELF

1. Notice that `IAisRepository` 's methods return `Task` but never say `async` . Why is `async` correctly absent from an interface?
2. The tests never touch a database. Which specific design decision makes that possible?
3. `AisValidator` has no interface and is a static class. Given everything above, why is that acceptable here — and what property of that class makes it so?

10 • BackgroundService — code that runs forever

The concept

A controller is *reactive*: a request arrives, it runs, it stops. Some work is not like that. It needs to start when the application starts and keep running until the application stops, with nobody prompting it.

ASP.NET Core calls this a **hosted service**. `BackgroundService` is the convenient base class: you override one method, and the framework handles starting, stopping, and shutdown signalling.

```
public class AisIngestionService : BackgroundService
{
    protected override async Task ExecuteAsync(CancellationToken stoppingToken)
    {
        // runs once, at start-up.
        // whatever you do here IS the service.
        // when this method returns, the service is finished.
    }
}
```

Registered with one line in `Program.cs` :

```
builder.Services.AddHostedService<AisIngestionService>(C);
```

From then on the host starts it automatically. Nothing else in the application refers to it — which is why you will not find `AisIngestionService` mentioned anywhere except that single registration.

The `CancellationToken` is the important part

The parameter `stoppingToken` is how the application asks your infinite loop to stop — on Ctrl+C, on `docker stop`, on an IIS recycle. Cooperative cancellation is a convention, not a force: **nothing can kill your loop; the token only asks**. You must honour it, in two ways.

```
// 1. Check it in every loop condition
while (!stoppingToken.IsCancellationRequested)
{
    ...
}

// 2. Pass it down to every await that accepts one
await ws.ConnectAsync(new Uri(_options.Url), stoppingToken);
await Task.Delay(TimeSpan.FromSeconds(delay), stoppingToken);
```

The second is the one beginners omit, and it matters more than the first. A loop that only checks the flag between iterations cannot stop while it is sitting inside a 30-second `Task.Delay`. Pass the token in, and that delay is abandoned the instant cancellation is requested. This is the difference between shutting down in 5 milliseconds and hanging for half a minute.

CANCELLATION THROWS — AND THAT IS NOT A BUG

When a token is cancelled, awaits that received it throw `OperationCanceledException`. This is normal shutdown, not failure, so it must be distinguished from a genuine error:

```
catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
{
    break; // clean stop - no error log
}
catch (Exception ex)
{
    _logger.LogError(ex, ...); // real failure - reconnect
}
```

The `when (...)` clause is an **exception filter**. It says: catch this only if the token really was cancelled. Without it you would swallow a genuine `OperationCanceledException` caused by, say, a network timeout, and silently treat a fault as a tidy shutdown.

ANALOGY — THE NIGHT WATCHMAN

He starts his round when the building opens and walks it continuously — nobody calls him, nobody schedules him. That is `ExecuteAsync`.

At closing time the manager rings a bell. The bell does not physically remove him; it asks him to finish up. If he is mid-corridor he hears it and returns. If he has his headphones on — a `Task.Delay` without the token — he keeps walking, and everyone waits for him.

The bell is the `CancellationToken`. Passing it into every await is taking the headphones off.

CHECK YOURSELF

1. What happens to the service when `ExecuteAsync` returns normally? What happens if it throws?
2. You remove `stoppingToken` from `await Task.Delay(TimeSpan.FromSeconds(30), stoppingToken)`. Describe precisely what a user pressing Ctrl+C now experiences.
3. Why is `catch (OperationCanceledException)` without the `when` filter a worse piece of code than the version with it?

The code, file by file

Every file in `InfraAis`, in the order that makes it easiest to learn: start-up, then configuration, then the data shapes, then the logic that moves data between them.

11 • `Program.cs` — the composition root

Thirty-four lines that define the entire application. Nothing here does any work; it assembles the machine and switches it on. Read it and you know what the program is made of.

InfraAis/Program.cs — complete

```
1 using InfraAis.Options;
2 using InfraAis.Services;
3 using InfraAis.Repositories;
4 var builder = WebApplication.CreateBuilder(args);
5
6 builder.Services.AddControllers();
7 builder.Services.AddEndpointsApiExplorer();
8 builder.Services.AddSwaggerGen();
9 builder.Services.AddOptions<AisStreamOptions>
10 ().Bind(builder.Configuration.GetSection(AisStreamOptions.SectionName));
11 builder.Services
12     .AddOptions<IngestionOptions>()
13     .Bind(builder.Configuration.GetSection(IngestionOptions.SectionName));
14 builder.Services.AddHostedService<AisIngestionService>();
15 builder.Services
16     .AddOptions<AisDbOptions>()
17     .Bind(builder.Configuration.GetSection(AisDbOptions.SectionName));
18 builder.Services.AddScoped<IAisRepository, AisRepository>();
19 builder.Services.AddScoped<IVesselIdentifierResolver, VesselIdentifierResolver>();
20
21 var app = builder.Build();
22
23 if (app.Environment.IsDevelopment())
24 {
25     app.UseSwagger();
26     app.UseSwaggerUI();
27 }
28 app.UseHttpsRedirection();
29
30 app.UseAuthorization();
31
32 app.MapControllers();
```

33

34 `app.Run();`

CONCEPT — TOP-LEVEL STATEMENTS

Where is `class Program`? Where is `static void Main`? Since C# 9 you may write statements directly in one file and the compiler generates the class and `Main` for you. `args` is still available; it is a parameter of the invisible `Main`. This is why the file looks like a script.

Line by line

line 4 - `WebApplication.CreateBuilder(args)`

Does far more than its length suggests. It creates the DI container, sets up logging to the console, determines the environment from `ASPNETCORE_ENVIRONMENT`, and loads the configuration stack described in Chapter 8 — `appsettings.json`, then the environment-specific file, then user secrets, then environment variables, then `args`. Everything that follows depends on this line having already run.

line 6 - `AddControllers()`

Registers the MVC machinery and scans the assembly for classes marked `[ApiController]`. This is how `VesselsController` is discovered — you never name it anywhere.

lines 7-8 - `AddEndpointsApiExplorer` / `AddSwaggerGen`

Swagger. The first exposes metadata about your endpoints; the second turns that metadata into an OpenAPI document. Together they give you the interactive test page at `/swagger`.

line 9 - `AddOptions<AisStreamOptions>().Bind(...)`

The options pattern from Chapter 8. Reads the `"AisStream"` section into an `AisStreamOptions` object and registers it as `IOptions<AisStreamOptions>`.

line 13 - `AddHostedService<AisIngestionService>()`

The single most consequential line in the file. It says: when this application starts, also start that worker and keep it running. Delete this line and the API still functions perfectly — it simply serves data that never gets any newer.

lines 17-18 - `AddScoped<interface, implementation>()`

The two recipes the container needs for the read side. Scoped means one instance per HTTP request (Chapter 7).

line 20 - `builder.Build()`

The dividing line of the file. Before it you are *registering* services; after it the container is sealed and you are *configuring the request pipeline*. You cannot add services after this point.

lines 22-26 - `if (IsDevelopment())`

Swagger only in development. A sound default: the interactive UI documents every endpoint and shape of your API, which is a gift to a developer and a starting point for an attacker.

lines 28-32 - the middleware pipeline

Order is significant here in a way it is not above. Each `Use...` call adds a layer that every request passes through in sequence. HTTPS redirection first, then authorization, then routing to a controller. Reversing the last two would attempt authorization after the request had already been handled.

line 34 – `app.Run()`

Starts the web server and **blocks until shutdown**. Every subsequent line would be unreachable. This call is also what triggers hosted services to start, and what signals the `CancellationToken` when the process is asked to stop.

ANALOGY — TWO HALVES OF A KITCHEN

Lines 6-18 are **stocking the kitchen**: deciding which ingredients and equipment will be available, and in what quantity. Lines 22-32 are **writing the recipe**: the ordered steps every dish must go through.

You can stock ingredients in any order — flour before eggs makes no difference. You cannot bake before you mix. That is exactly why registration order is irrelevant above line 20 and critical below it.

An observation worth making

Line 9 is written on one long line while lines 10-12 and 14-16 do the same thing across three. It is cosmetic, but consistency in a composition root has practical value: this file is the map of the application, and a tidy map is read more often. Grouping the three `AddOptions` calls together, then the two `AddScoped` calls, then the hosted service, would let a newcomer take in the whole system at a glance.

CHECK YOURSELF

1. You move line 13 to sit above line 6. Does the application still behave identically? Why?
2. You swap lines 30 and 32. Does *that* still behave identically? Why is your answer different?
3. You comment out line 13 and run the app. Which endpoints still work, and what changes about the data they return over the following hour?

12 • Options/ — typed configuration

Three small classes, one per configuration section. Chapter 8 covered the mechanism; here is what each one actually controls.

AisStreamOptions — the feed

InfraAis/Options/AisStreamOptions.cs

```
public class AisStreamOptions
{
    public const string SectionName = "AisStream";

    public string Url { get; set; } = "";
    public string ApiKey { get; set; } = "";
    public BoundingBoxOptions BoundingBox { get; set; } = new();
    public string[] FilterMessageTypes { get; set; } = [];

    public bool DangerouslyAcceptInvalidFeedCertificate { get; set; }
}

public class BoundingBoxOptions
{
    public double MinLat { get; set; }
    public double MinLon { get; set; }
    public double MaxLat { get; set; }
    public double MaxLon { get; set; }
}
```

BoundingBox — a nested object

Binding is recursive. A nested JSON object maps to a nested class automatically, provided the property names match. Note `= new()`: without it the property would be null when the section is absent, and `_options.BoundingBox.MinLat` would throw. Initialising it means "worst case, an empty box" rather than "worst case, a crash".

FilterMessageTypes = []

C# 12 collection-expression syntax for an empty array — the modern spelling of `Array.Empty<string>()`. Same defensive reasoning: an empty array is safe to iterate, null is not.

DangerouslyAcceptInvalidFeedCertificate

Note the name. It is deliberately alarming, because it disables TLS certificate validation. Good naming is a security control: nobody sets this by accident, and it cannot appear in a code review without provoking a question. Chapter 16 shows the second safeguard that backs it up.

IngestionOptions — the tuning knobs

Setting	Default	Controls
<code>FutureSkewMinutes</code>	5	How far ahead of our clock a message timestamp may be before we reject it as nonsense.
<code>ReconnectBaseDelaySeconds</code>	1	The first pause after a dropped connection.
<code>ReconnectMaxDelaySeconds</code>	30	The ceiling that exponential backoff doubles up towards.
<code>SummaryIntervalSeconds</code>	30	How often the throughput line is logged and counters reset.

AisDbOptions — a special case worth pausing on

InfraAis/Options/AisDbOptions.cs

```
public class AisDbOptions
{
    public const string SectionName = "ConnectionStrings";
    public string AisDb { get; set; } = "";
}
```

Look closely: the section name is `"ConnectionStrings"`, which is a *built-in* ASP.NET Core section, and the property is named after the key inside it.

```
"ConnectionStrings": {
  "AisDb": "Server=localhost\\SQLEXPRESS;Database=AisTrackingDB;..."
}
```

So `AisDbOptions.AisDb` maps to `ConnectionStrings:AisDb`. The framework offers a shorthand for this — `builder.Configuration.GetConnectionString("AisDb")` — but binding it as options instead keeps the repository free of any dependency on `IConfiguration`, which is a cleaner boundary and easier to test.

THE FAILURE MODE HIDING HERE

The default is `""`. If the connection string is missing or misspelled, `AisDb` is an empty string, the application starts perfectly happily, and the first database write fails with `ArgumentException: The value's length for key 'data source' must be greater than zero`.

You will see this as a mysterious error inside the ingestion loop several seconds after start-up, with nothing pointing at configuration. This is precisely the case `ValidateOnStart()` exists to prevent — see Chapter 23.

CHECK YOURSELF

1. Why does `BoundingBox` have `= new()` when `Url` merely has `= ""` ? What differs about the consequences?
2. Your connection string key is misspelled as `"AisDB"` . At what moment does the program fail, and what will the message be about?
3. `SectionName` is declared `const` on each options class rather than being typed as a literal in `Program.cs` . Name one concrete bug that habit prevents.

13 • Models/AisMessage.cs — the shape of the feed

These classes are not yours. They are a C# mirror of the JSON `aisstream.io` chooses to send. Their job is to be deserialized into, and nothing else — no methods, no validation, no logic.

What arrives on the wire

```
{
  "MessageType": "PositionReport",
  "MetaData": {
    "MMSI": 255805678,
    "ShipName": "BUXCLIFF",
    "time_utc": "2024-05-20 09:21:31.781972101 +0000 UTC"
  },
  "Message": {
    "PositionReport": {
      "UserID": 255805678,
      "Latitude": 51.4,
      "Longitude": 3.1,
      "Sog": 15.6,
      "Cog": 271.2,
      "TrueHeading": 271,
      "NavigationalStatus": 0,
      "RateOfTurn": 0,
      "PositionAccuracy": true,
      "Valid": true
    }
  }
}
```

Note the envelope structure: a `MessageType` discriminator, a `MetaData` block, and a `Message` block containing *one* populated property named after the type. That shape dictates the classes.

InfraAis/Models/AisMessage.cs — the envelope

```
5 public class AisMessage
6 {
7     public string MessageType { get; set; } = "";
8     public MessageBody Message { get; set; } = new();
9     public MetaData MetaData { get; set; } = new();
10 }
11
12 public class MessageBody
13 {
14     public PositionReport? PositionReport { get; set; }
15     public ShipStaticData? ShipStaticData { get; set; }
16 }
```

lines 14-15 – both nullable

This is the type system telling the truth about the protocol. A message is *either* a position report *or* static data — never both. Whichever is not present deserializes to null. That is why `HandlePositionReport` opens with `if (pr is null) return;` rather than trusting the discriminator.

```
= "" and = new()
```

Non-nullable properties with defaults. Without them the compiler warns that a non-nullable property is never assigned. More practically, they mean a malformed message yields empty objects rather than nulls, so the switch on `MessageType` falls through to `default` instead of throwing.

The one attribute in the file, and why it is needed

InfraAis/Models/AisMessage.cs — lines 61-68

```
public class MetaData
{
    public long MMSI { get; set; }
    public string ShipName { get; set; } = "";

    [JsonPropertyName("time_utc")]
    public string TimeUtc { get; set; } = "";
}
```

Deserialization in this project is configured to be case-insensitive:

```
private static readonly JsonSerializerOptions _jsonOptions = new()
{
    PropertyNameCaseInsensitive = true
};
```

That handles `MMSI` → `Mmsi` and `Latitude` → `latitude`. But `time_utc` has an **underscore**, and no amount of case-insensitivity turns `time_utc` into `TimeUtc`. The attribute supplies the mapping explicitly.

THE SILENT FAILURE THIS PREVENTS

Remove that attribute and nothing breaks visibly. `System.Text.Json` does not complain about JSON properties it cannot match — it simply leaves `TimeUtc` as its default `""`. Then `TryParseTimestamp("")` returns false, every message is dead-lettered as "unparseable timestamp", and your `positions` table stays empty while the log fills with rejections.

This is the characteristic shape of a JSON mapping bug: **no exception, no error, just data quietly not arriving**. When a field is unexpectedly empty, suspect the name mapping first.

The sentinel values, in the raw

Look at `PositionReport` and notice that nothing is nullable:

```
public double Sog { get; set; }           // 102.3 means "unknown"
public double Cog { get; set; }           // 360   means "unknown"
public int TrueHeading { get; set; }      // 511   means "unknown"
public int RateOfTurn { get; set; }       // -128  means "unknown"
public bool Valid { get; set; }           // the feed's own quality flag
```

That is correct and intentional. **This class represents the wire format faithfully, including its warts**. The AIS protocol genuinely has no concept of null — it encodes "unknown" as a reserved number,

because it was designed for fixed-width radio frames.

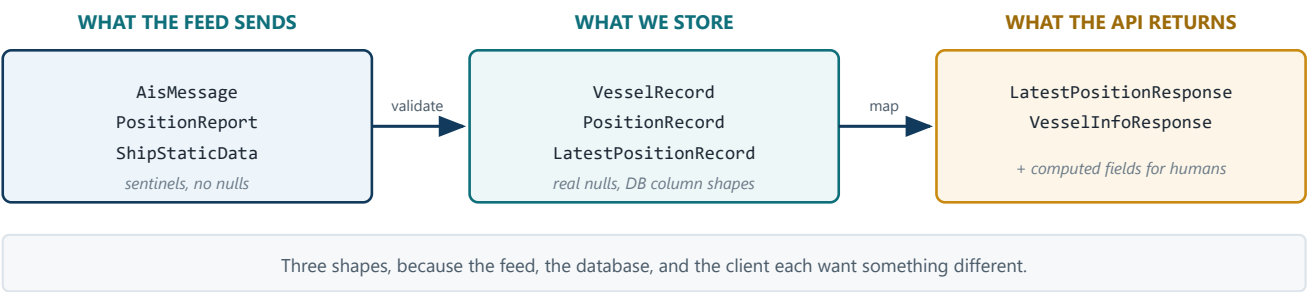
Converting those to real nulls is a separate job, done later, by `AisValidator`. Keeping the two apart is a good instinct worth naming: **parse faithfully first, interpret second**. If you made `Sog` nullable here, you would have to embed AIS domain knowledge into your deserialization layer, and you would lose the ability to dead-letter the raw message exactly as it arrived.

CHECK YOURSELF

1. The feed renames `time_utc` to `timestamp_utc` in a new API version. What breaks, what does the log show, and what does `SELECT COUNT(*) FROM positions` return an hour later?
2. Why are `PositionReport` and `ShipStaticData` both nullable inside `MessageBody`, when every message must contain one of them?
3. Argue against making `Sog` a `double?` in this class. What responsibility would move to the wrong layer?

14 • Models/ — records and responses

Seven model classes sit in this folder and they are easy to confuse. The organising insight is that they belong to **three different jobs**, and a class that tries to serve two of them will eventually break.



Storage records

`VesselRecord` and `PositionRecord` are the "ready to store" shapes. Compare a field across the two layers and the transformation becomes obvious:

Field	Feed (PositionReport)	Storage (PositionRecord)	Why it changed
Speed	<code>double Sog</code>	<code>decimal? Sog</code>	Nullable because 102.3 became a real null. <code>decimal</code> because the column is <code>DECIMAL(5,1)</code> .
Heading	<code>int TrueHeading</code>	<code>short? TrueHeading</code>	<code>short</code> matches SQL <code>SMALLINT</code> exactly; 0-359 never needs more.
Accuracy	<code>bool PositionAccuracy</code>	<code>bool? PositionAccuracy</code>	Nullable to mirror a nullable <code>BIT</code> column.

WHY DECIMAL AND NOT DOUBLE FOR SPEED?

`double` is binary floating point: fast, and unable to represent 0.1 exactly. `decimal` is base-10: slower, and exact for the values humans write down.

Latitude and longitude stay `double` here — they are physical measurements, the column is `FLOAT`, and tiny representation error is irrelevant next to GPS error. Speed and course become `decimal` because the columns are `DECIMAL(5,1)` and matching the CLR type to the SQL type avoids a silent conversion on every insert. **The rule of thumb: `double` for measurements, `decimal` for anything counted, priced, or stored in a decimal column.**

Response models, and why they are not just the records

It would be less code to return `LatestPositionRecord` straight from the controller. The project defines a separate `LatestPositionResponse` instead, and the two extra properties show why:

```
InfraAis/Models/LatestPositionResponse.cs
```

```

public class LatestPositionResponse
{
    public long Mmsi { get; set; }
    public int? Imo { get; set; }
    public string? Name { get; set; }
    public double Latitude { get; set; }
    public double Longitude { get; set; }
    public decimal? Sog { get; set; }
    public decimal? Cog { get; set; }
    public int? NavStatus { get; set; }
    public string NavStatusText { get; set; } = ""; // ← not in the database
    public DateTime TimestampUtc { get; set; }
    public double AgeSeconds { get; set; } // ← computed per request
}

```

NavStatusText

The database stores `nav_status = 0`. A human wants "Under way using engine". The translation happens in `NavStatusMapper` (Chapter 20) and lands here. Storing the text would duplicate it across millions of rows; computing it on read costs nothing.

AgeSeconds

Cannot be stored, because it changes every second. Computed at the moment of the request as `DateTime.UtcNow - MsgTimestampUtc`. This is the field that tells a caller whether they are looking at a live position or a ghost from six hours ago — arguably the most operationally useful value in the whole response.

CONCEPT — WHY SEPARATE A DTO FROM A STORAGE MODEL AT ALL

Three reasons, in increasing order of importance.

Shape. The client wants fields the database does not have, and does not want fields it does.

Stability. Your API is a public contract. If the response type *is* your storage type, renaming a database column silently changes the JSON every consumer parses. The separate class absorbs that shock.

Leakage. Sooner or later a storage model gains an internal field — a soft-delete flag, an ingestion batch id, an internal note. If it doubles as your response type, that field is now on the public internet and nobody noticed.

VesselIdentifier — a tiny class doing real work

InfraAis/Models/VesselIdentifier.cs — complete

```

public enum IdentifierType
{
    Mmsi,
    Imo
}

public class VesselIdentifier
{
    public IdentifierType Type { get; set; }
    public long Value { get; set; }
}

```

Thirteen lines, and the point of them is that `9074729` on its own is ambiguous. Is it an IMO or a badly formed MMSI? A bare `long` cannot say. This pairs the number with its meaning so that, once past the parsing step, **no later code has to guess**.

The `enum` is doing the same job the nullable annotations did in Chapter 4 — moving a fact out of your head and into the type system, where the compiler can enforce it. When you find yourself writing a comment like *"note: this long is an IMO"*, that is the signal to reach for a type instead.

CHECK YOURSELF

1. Why is `AgeSeconds` not a column in the `positions` table?
2. `Latitude` is `double` but `Sog` is `decimal`. Give the reason for each choice.
3. You return `LatestPositionRecord` directly from the controller and delete `LatestPositionResponse`. Name two things the API loses, and one future bug it invites.

15 • Validation/AisValidator.cs — the rules

The most important class in the project to understand, and the easiest to test — because it is **pure**. Every method takes values in and returns values out. No database, no network, no clock (with one exception we will come to), no state. Give it the same input twice and it gives the same answer twice.

That purity is why 52 of the project's 80 tests point at this one file, and why they run in milliseconds.

Two kinds of method, and the difference matters

Kind	Signature shape	Meaning
Validators	<code>bool IsValidXxx(...)</code>	"Is this usable at all?" A <code>false</code> means reject the whole message — it goes to the dead-letter table and no row is stored.
Normalizers	<code>T? NormalizeXxx(...)</code>	"Clean this field." A <code>null</code> means store NULL in this column — the message is still good, we simply do not know this one value.

Keep that distinction sharp. Confusing them is how you either throw away good data or fill a database with 102.3-knot container ships.

The straightforward rules

InfraAis/Validation/AisValidator.cs — lines 6-37

```
6 public static bool IsValidMmsi(long mmsi)
7 {
8     return mmsi >= 100_000_000 && mmsi <= 999_999_999;
9 }
10
11 public static bool IsValidPosition(double lat, double lon)
12 {
13     return lat >= -90 && lat <= 90
14         && lon >= -180 && lon <= 180;
15 }
16 public static decimal? NormalizeSog(double sog)
17 {
18     if (sog < 0 || sog == 102.3) return null;
19     return (decimal)sog;
20 }
21
22 public static decimal? NormalizeCog(double cog)
23 {
24     return (cog < 0 || cog >= 360) ? null : (decimal)cog;
25 }
26 public static int? NormalizeNavStatus(int navStatus)
27 {
28     return (navStatus < 0 || navStatus > 15) ? null : navStatus;
29 }
30 public static short? NormalizeRateOfTurn(int rateOfTurn)
31 {
32     return (rateOfTurn < -127 || rateOfTurn > 127) ? null : (short)rateOfTurn;
```

```

33 }
34 public static short? NormalizeHeading(int heading)
35 {
36     return (heading < 0 || heading > 359) ? null : (short)heading;
37 }

```

line 8 – the underscores

`100_000_000` is C#'s digit separator. It compiles identically to `100000000` and exists purely so a human can see at a glance that it is nine digits. Use it for any literal above four digits.

lines 13-14 – no mention of 91 or 181

The sentinel for "no fix" is latitude 91, longitude 181. Notice the code does not special-case them: a plain range check rejects them automatically, because 91 is not a latitude. **Handling a special case by making it fall out of the general rule is better than handling it explicitly** — there is no second rule to forget to update.

line 18 – `sog == 102.3`

Floating-point equality, which is normally a red flag. It works here because the JSON text `102.3` and the C# literal `102.3` both convert to the identical bit pattern. It is fragile rather than broken — Chapter 23 revisits it.

line 24 – `cog >= 360`

Note `>=`, not `==`. Course is 0-359.9, so 360 is the sentinel and anything above is nonsense. One comparison covers both.

line 32 – rate of turn

The AIS sentinel is `-128`, and the valid range is -127 to +127. Again, the range check subsumes the sentinel. The cast to `short` is safe precisely *because* it happens after the range check.

The IMO checksum

An IMO number's seventh digit is a check digit derived from the first six. This catches transcription errors — and, in this feed, transponders broadcasting garbage.

InfraAis/Validation/AisValidator.cs — lines 38-55

```

38 public static bool IsValidImo(int imo)
39 {
40     if (imo < 1_000_000 || imo > 9_999_999)
41         return false;
42     int sum = 0;
43     int weight = 2;
44     int remaining = imo / 10;
45     for (int i = 0; i < 6; i++)
46     {
47         int digit = remaining % 10;
48         sum += digit * weight;
49         remaining /= 10;
50         weight++;
51     }
52
53     int checkDigit = imo % 10;
54     return sum % 10 == checkDigit;
55 }

```

The rule: multiply the six leading digits by 7, 6, 5, 4, 3, 2 from left to right, sum, and take the last digit. That must equal the seventh digit.

The code walks the number **right to left** instead — because `% 10` peels digits off the right cheaply — so the weights count *up* from 2 as it moves left. Same arithmetic, less code. Worked through with the real IMO **9074729**:

i	remaining	digit	weight	digit × weight	sum
—	907472	—	2	—	0
0	907472	2	2	4	4
1	90747	7	3	21	25
2	9074	4	4	16	41
3	907	7	5	35	76
4	90	0	6	0	76
5	9	9	7	63	139

`139 % 10 = 9`. The check digit is `9074729 % 10 = 9`. They match, so the number is valid — and indeed 9074729 is a real ship.

WHY THIS MATTERS MORE THAN IT LOOKS

Recall from Chapter 1 that the IMO is the *permanent* identifier, and from Chapter 21 that `UX_vessels_imo` is a unique index. Store a corrupt IMO and you have not merely recorded one bad field — you have claimed that hull number belongs to this vessel, and the next real ship with that IMO cannot be inserted.

That is why the calling code stores `null` rather than the bad value:

```
int? imo = AisValidator.IsValidImo(sd.ImoNumber) ? sd.ImoNumber : null;
```

A missing identifier is a gap. A wrong identifier is corruption. Prefer the gap.

The Go timestamp — the most interesting method in the file

The feed is written in Go, and it sends timestamps in Go's default format:

```
2024-05-20 09:21:31.781972101 +0000 UTC
```

.NET's default parser refuses this, for two independent reasons. First, there are **nine** fractional-second digits; .NET's `DateTime` has 100-nanosecond resolution and supports at most seven. Second, there is a literal `UTC` suffix *after* the numeric offset — which is redundant, since `+0000` already said so, and which no standard format expects.

```

56 public static bool TryParseTimestamp(string raw, out DateTime utc)
57 {
58     utc = default;
59     if (string.IsNullOrEmpty(raw))
60         return false;
61
62     var cleaned = raw.Replace(" UTC", "").Trim();
63
64     cleaned = TrimFraction(cleaned);
65
66     return DateTime.TryParseExact(
67         cleaned,
68         "yyyy-MM-dd HH:mm:ss.FFFFFFFF zzz",
69         CultureInfo.InvariantCulture,
70         DateTimeStyles.AdjustToUniversal | DateTimeStyles.AssumeUniversal,
71         out utc);
72 }
73
74 private static string TrimFraction(string s)
75 {
76     int dot = s.IndexOf('.');
77     if (dot == -1) return s;
78     int end = dot + 1;
79     while (end < s.Length && char.IsDigit(s[end]))
80         end++;
81
82     int fracLen = end - (dot + 1);
83     if (fracLen <= 7) return s;
84
85     return s.Substring(0, dot + 1 + 7) + s.Substring(end);
86 }

```

line 56 – the Try-pattern

A .NET convention: return `bool` for success, hand the result back through an `out` parameter. The alternative — throwing on bad input — would be wrong here, because bad timestamps are *expected* from a noisy radio feed. Exceptions are for the exceptional; this is routine.

line 58 – utc = default

Required by the compiler: an `out` parameter must be assigned on every path, including the early `return false`. `default` for `DateTime` is `0001-01-01`.

line 62 – stripping " UTC"

Removes the trailing suffix. `Replace` is global rather than anchored to the end, which is slightly loose, but no legitimate AIS timestamp contains " UTC" anywhere else.

lines 76-80 – locating the fraction

Find the decimal point, then walk forward over digits until they run out. `end` now indexes the first character *after* the fractional digits — the space before `+0000`.

line 83 – the early exit

Seven or fewer digits is already legal, so return the string untouched. This is what makes the method safe against a future feed change that shortens the precision.

line 85 – the surgery

Keep everything up to and including the seventh fractional digit, then skip the excess and re-attach the remainder. `...31.781972101 +0000` becomes `...31.7819721 +0000`. Note that this **truncates rather than rounds** — losing at most 100 nanoseconds, which for a ship doing 15 knots is roughly one micrometre of travel.

line 68 – the format string

`FFFFFFF` (capital F) means "up to seven fractional digits, all optional". Capital-F also makes the preceding decimal separator optional, so a timestamp with no fraction at all still parses. `zzz` matches the `+0000` offset.

line 69 – `InvariantCulture`

Essential, not decorative. On a machine with German regional settings the decimal separator is a comma; without this argument the parse would fail in production and succeed on your laptop. Any time you parse machine-generated text, pass `InvariantCulture`.

line 70 – the two style flags

`AssumeUniversal`: if no offset were present, treat the value as UTC rather than local time.

`AdjustToUniversal`: having parsed the offset, convert the result to UTC and mark it as such. Together they guarantee the `DateTime` that comes out is unambiguously UTC — which matters, because it is about to be compared against `DateTime.UtcNow` and written to a `DATETIME2` column that stores no time zone.

The one impure method

```
87 public static bool IsNotFutureSkewed(DateTime utc, int maxSkewMinutes)
88 {
89     return utc <= DateTime.UtcNow.AddMinutes(maxSkewMinutes);
90 }
```

This one reads the system clock, so it is not pure — the same input can give different answers a minute apart. That is a small crack in an otherwise clean class, and you can see its effect in the tests, which have to construct their inputs relative to `DateTime.UtcNow` rather than using fixed values.

The purpose is real, though: a transponder with a badly set clock can report a position dated next Tuesday. Storing it would corrupt every "latest position" query, since ordering is by timestamp — one future-dated row outranks every genuine report until Tuesday actually arrives. The five-minute tolerance allows for ordinary clock drift while catching the absurd.

CHECK YOURSELF

1. A message has latitude 91. Which method rejects it, and where in the code is 91 mentioned?
2. Work out by hand whether 9704612 is a valid IMO number. Show the sum.
3. What exactly goes wrong if line 69 passes `CultureInfo.CurrentCulture` instead, and on which machines would you fail to notice?
4. Why is a future-dated position more damaging than a past-dated one, given how "latest position" is computed?

16 • `Services/AisIngestionService.cs` — the engine room

The largest and most demanding file in the project, and the one where the concepts from Part II all show up at once. Take it in six pieces.

16.1 • The fields and the constructor

`InfraAis/Services/AisIngestionService.cs` — lines 11-40

```
11 public class AisIngestionService : BackgroundService
12 {
13     private readonly ILogger<AisIngestionService> _logger;
14     private readonly AisStreamOptions _options;
15
16     private readonly IngestionOptions _ingestion;
17     private readonly IServiceScopeFactory _scopeFactory;
18     private readonly IHostEnvironment _environment;
19     private long _received;
20     private long _stored;
21     private long _rejected;
22     private long _staticStored;
23     private long _reconnects;
24     public AisIngestionService(
25         ILogger<AisIngestionService> logger,
26         IOptions<AisStreamOptions> options,
27         IOptions<IngestionOptions> ingestion,
28         IServiceScopeFactory scopeFactory,
29         IHostEnvironment environment)
30     {
31         _logger = logger;
32         _options = options.Value;
33         _ingestion = ingestion.Value;
34         _scopeFactory = scopeFactory;
35         _environment = environment;
36     }
37     private static readonly JsonSerializerOptions _jsonOptions = new()
38     {
39         PropertyNameCaseInsensitive = true
40     };
```

line 11 – : `BackgroundService`

Inheritance. Everything from Chapter 10 applies: override `ExecuteAsync`, honour the token.

lines 13-18 – `readonly` dependencies

`readonly` means assignable only in the constructor. It is a promise that these references never change for the life of the object — worth applying to every injected dependency, since it turns a whole class of accidental reassignment into a compile error.

lines 19-23 – `mutable` counters

Not `readonly`, because these change constantly. They are the observability state (§16.6).

lines 32-33 – `.Value`

Unwrapping `IOptions<T>`, exactly as Chapter 8 described. Because this is a singleton reading `IOptions`, these settings are frozen at start-up for the life of the process.

line 28 – IServiceScopeFactory

The solution to the scope trap from Chapter 7. Not a repository — a means of making scopes in which repositories can briefly live.

lines 37-40 – static readonly options

`static` so a single instance is shared by every call rather than allocated per message.

`JsonSerializerOptions` is expensive to construct and internally caches its reflection metadata; creating one per message is a well-known .NET performance mistake. This is the right pattern.

16.2 · `ExecuteAsync` — the supervisor

This method's entire job is to make sure `ConnectAndListenAsync` keeps running, no matter how often the network fails.

InfraAis/Services/AisIngestionService.cs — lines 41-70

```
41 protected override async Task ExecuteAsync(CancellationToken stoppingToken)
42 {
43     _ = RunSummaryLoopAsync(stoppingToken);
44     var delay = _ingestion.ReconnectBaseDelaySeconds;
45
46     while (!stoppingToken.IsCancellationRequested)
47     {
48         try
49         {
50             await ConnectAndListenAsync(stoppingToken);
51
52             delay = _ingestion.ReconnectBaseDelaySeconds;
53         }
54         catch (OperationCanceledException) when (stoppingToken.IsCancellationRequested)
55         {
56             break;
57         }
58         catch (Exception ex)
59         {
60             _reconnects++;
61             _logger.LogError(ex, "Feed connection failed. Reconnecting in {Delay}s...", delay);
62
63             await Task.Delay(TimeSpan.FromSeconds(delay), stoppingToken);
64
65             delay = Math.Min(delay * 2, _ingestion.ReconnectMaxDelaySeconds);
66         }
67     }
68
69     _logger.LogInformation("AIS worker stopping.");
70 }
```

line 43 – the discard

`_ =` means "start this and deliberately ignore the Task". It launches the summary loop as a second concurrent activity. The discard silences the compiler warning about an un-awaited call. **This is fire-and-forget, and it has a sharp edge** — see §16.6.

line 46 – the outer loop

Runs until shutdown. Its purpose is resilience: an exception anywhere inside `ConnectAndListenAsync` is caught here rather than killing the service.

line 50 – the real work

One call, which under normal conditions does not return for hours. When it does return, the connection has closed cleanly.

line 52 – resetting the backoff

After a successful session, forget any accumulated penalty. Without this, one bad hour in the morning would leave you reconnecting at 30-second intervals all afternoon.

lines 54–57 – the shutdown path

The exception filter from Chapter 10. Cancellation is not failure: `break` out and exit quietly, with no error logged and no reconnect attempted.

lines 58–66 – the failure path

Any other exception means the feed broke. Count it, log it *with the exception object* so the stack trace survives, wait, and double the wait for next time.

line 63 – token passed to Delay

The detail from Chapter 10 that makes shutdown instant instead of taking up to 30 seconds.

line 65 – the backoff formula

`Math.Min(delay * 2, max)` gives 1, 2, 4, 8, 16, 30, 30, 30... The doubling backs off fast enough to stop hammering a dead server; the cap stops it backing off to hours and missing the recovery.

ANALOGY — REDIALLING A BUSY NUMBER

The line is engaged. You do not redial every half-second for an hour — you would achieve nothing and annoy everybody. You wait a second, then two, then four. But you also do not double forever, or you would still be waiting three hours after they hung up. So you cap it at thirty seconds and keep trying steadily.

And when you finally get through and have a proper conversation, you forget the whole escalation. Next time the line is busy, you start again at one second. That is line 52.

A GAP IN THIS LOGIC WORTH SPOTTING

Trace what happens when the server accepts the connection and then immediately closes it — a common symptom of an invalid API key or a rate limit.

`ConnectAndListenAsync` returns **normally** (see line 117: it `return`s on a close frame, it does not throw). So line 52 resets the delay, the `while` loop goes round, and it reconnects **immediately, with no delay at all**. If the server keeps closing instantly, this becomes a tight reconnect loop hammering the provider hundreds of times a second.

The backoff only protects the *exception* path. The clean-close path has no protection. Chapter 23 proposes the fix.

16.3 · ConnectAndListenAsync — the connection

The TLS escape hatch

```
76     if (_options.DangerouslyAcceptInvalidFeedCertificate && _environment.IsDevelopment())
77     {
78         _logger.LogWarning(
79             "TLS certificate validation is DISABLED for the AIS feed (development only). " +
80             "Unset AisStream:DangerouslyAcceptInvalidFeedCertificate once the upstream
certificate is renewed.");
81         ws.Options.RemoteCertificateValidationCallback = (_, _, _, _) => true;
82     }
83     else if (_options.DangerouslyAcceptInvalidFeedCertificate)
84     {
85         _logger.LogError(
86             "AisStream:DangerouslyAcceptInvalidFeedCertificate is set but the environment is
{Environment}, not Development. " +
87             "Certificate validation stays ENABLED.", _environment.EnvironmentName);
88     }
```

This is a nicely constructed piece of defensive code and worth studying as a pattern.

Turning off certificate validation means accepting *any* certificate — including one presented by an attacker intercepting your traffic. Sometimes you genuinely need it, when an upstream provider lets their certificate expire and you have work to do.

The safeguard is **two independent conditions**. The flag must be set *and* the environment must be Development. Setting the flag in production does not weaken security; it produces an error log saying so. A single condition would have let one stray environment variable expose production traffic.

line 81 - `(_, _, _, _) => true`

A lambda whose four parameters are all discards — "I do not care about the certificate, the chain, or the errors; approve everything." The discards make it visually obvious that no checking occurs.

The subscription

```
91     await ws.ConnectAsync(new Uri(_options.Url), stoppingToken);
...
94     var box = _options.BoundingBox;
95     var subscription = new
96     {
97         APIKey = _options.ApiKey,
98         BoundingBoxes = new[] { new[] { new[] { box.MinLat, box.MinLon }, new[] { box.MaxLat,
box.MaxLon } } },
99         FilterMessageTypes = _options.FilterMessageTypes
100     };
101     var json = JsonSerializer.Serialize(subscription);
102     var sendBytes = Encoding.UTF8.GetBytes(json);
103     await ws.SendAsync(sendBytes, WebSocketMessageType.Text, endOfMessage: true,
stoppingToken);
```

line 95 - an anonymous type

`new { ... }` with no class name. The compiler generates a throwaway type with those properties. Ideal here: this object exists only to be serialized once and discarded, so naming it would add a file for no benefit.

line 98 – the triple array

Genuinely awkward, and it is the provider's schema, not a mistake. The protocol wants a *list of boxes*, each box being a *list of two corners*, each corner a *list of two numbers*. Hence `[[[minLat,minLon], [maxLat,maxLon]]]` — one box, two corners, two numbers each.

line 103 – `endOfMessage: true`

A **named argument**. The parameter is a bare `bool`, and `SendAsync(bytes, type, true, token)` would be unreadable at the call site. Naming it documents the call in place. Use this habit for every boolean argument you pass.

THE THREE-SECOND RULE

`aisstream.io` requires the subscription message within three seconds of connecting, or it closes the socket. That is why nothing else happens between lines 91 and 103. If you ever add work in there — a database check, a lookup — you will produce a connection that mysteriously drops immediately.

Reassembling messages from frames

```
106     var buffer = new byte[8192];
107     while (!stoppingToken.IsCancellationRequested && ws.State == WebSocketState.Open)
108     {
109         using var ms = new MemoryStream();
110         WebSocketReceiveResult result;
111         do
112         {
113             result = await ws.ReceiveAsync(buffer, stoppingToken);
114             if (result.MessageType == WebSocketMessageType.Close)
115             {
116                 _logger.LogWarning("Server closed the connection.");
117                 return;
118             }
119             ms.Write(buffer, 0, result.Count);
120         }
121         while (!result.EndOfMessage);
122
123         var message = Encoding.UTF8.GetString(ms.ToArray());
```

CONCEPT — WHY A MESSAGE NEEDS REASSEMBLING AT ALL

A WebSocket does not deliver whole messages; it delivers **frames**. A large message arrives as several frames, and only the last one has `EndOfMessage == true`.

So the buffer is 8 KB, and if a message exceeds that it arrives in pieces. The inner `do...while` keeps receiving into a `MemoryStream` until the flag says the message is complete. Only then is the accumulated bytes converted to a string.

Skip this and you get an intermittent bug of the worst kind: everything works in testing, and occasionally in production a longer-than-usual message is cut in half, producing a JSON parse error on a message that was perfectly valid. It would fail more often for ships with long names — which is a maddening thing to debug.

line 107 – two conditions

Stop on shutdown, and stop if the socket leaves the Open state for any reason.

line 109 – a fresh MemoryStream per message

Inside the loop, with `using`, so each message starts empty and the buffer is released afterwards. Hoisting this out of the loop to "save an allocation" would concatenate every message into one ever-growing blob — a tempting and disastrous optimisation.

lines 114–118 – the close frame

A close frame is a normal protocol event, not an error, so the method `return`s rather than throwing. As noted in §16.2, that choice is what bypasses the backoff.

line 123 – decode once, at the end

UTF-8 characters can span byte boundaries. Decoding each frame separately could split a multi-byte character in two and corrupt it. Accumulate bytes, decode once.

Dispatching on message type

```
124     _received++;
125     try
126     {
127         var ais = JsonSerializer.Deserialize<AisMessage>(message, _jsonOptions);
128
129         switch (ais?.MessageType)
130         {
131             case "PositionReport":
132                 await HandlePositionReport(ais, message);
133                 break;
134             case "ShipStaticData":
135                 await HandleStaticData(ais, message);
136                 break;
137             default:
138                 _logger.LogDebug("Skipping type: {Type}", ais?.MessageType);
139                 break;
140         }
141     }
142     catch (JsonException ex)
143     {
144         _logger.LogWarning("Bad message skipped: {Error}", ex.Message);
145     }
```

line 129 – switch on a string

Legal in C#, and the natural fit for a JSON type discriminator. The `?.` guards against `Deserialize` returning null for the literal input `"null"`.

lines 137–139 – the default case

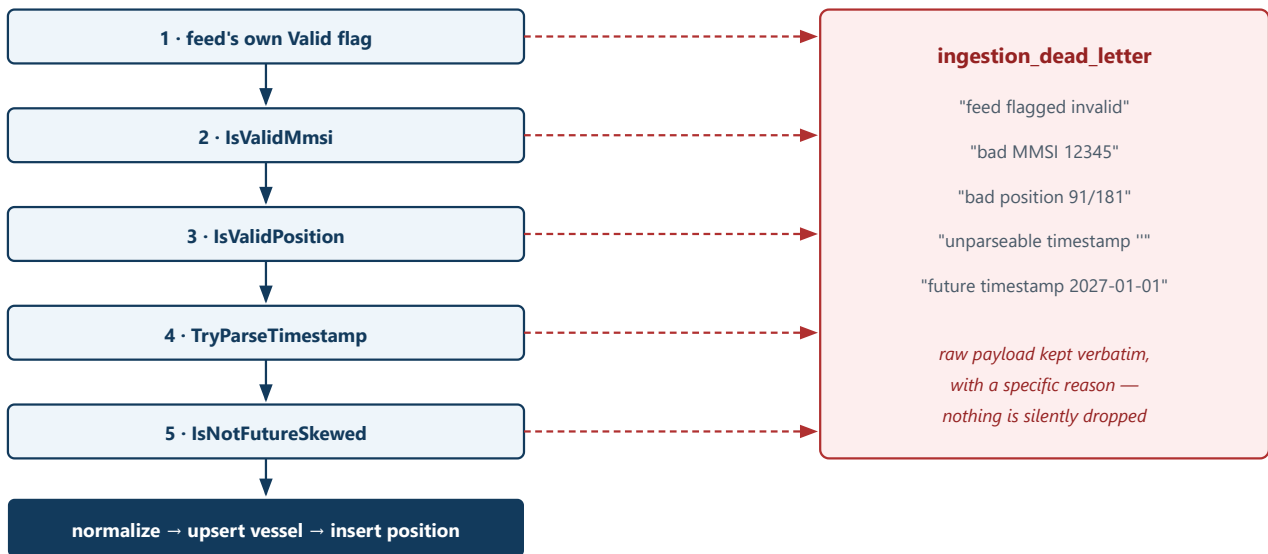
Other AIS types (there are 27) are logged at Debug and dropped. Note they are **not** dead-lettered — an unwanted type is not a bad message.

lines 142–145 – narrow catch, inside the loop

Two decisions worth naming. **Narrow:** only `JsonException` is caught, so a database failure propagates up to the supervisor and triggers a reconnect rather than being silently swallowed. **Inside the loop:** one malformed message is skipped and the loop continues; it never takes down the connection.

16.4 · HandlePositionReport — the gauntlet

Five checks in sequence. Every one follows an identical shape, and the repetition is the point — it is obvious, auditable, and each rejection reason is specific enough to diagnose from the dead-letter table.



The shape of every check, taken from lines 164-170:

```
if (!AisValidator.IsValidMmsi(pr.UserID))
{
    await repo.InsertDeadLetterAsync(rawPayload, $"bad MMSI {pr.UserID}");
    _rejected++;
    _logger.LogWarning("REJECT: bad MMSI={Mmsi}", pr.UserID);
    return;
}
```

InsertDeadLetterAsync(rawPayload, ...)

The **original, untouched** JSON string is stored — not the parsed object. That is deliberate: when you later investigate why a vessel is missing, you need what actually arrived, not your interpretation of it.

(\$"bad MMSI {pr.UserID}")

String interpolation putting the offending value into the reason. `SELECT reason, COUNT(*) FROM ingestion_dead_letter GROUP BY reason` then tells you exactly which rule is firing and how often.

return

Abandon this message entirely. Nothing is written to `vessels` or `positions`.

Then the storing

```
195     var sog = AisValidator.NormalizeSog(pr.Sog);
196     var cog = AisValidator.NormalizeCog(pr.Cog);
197     var heading = AisValidator.NormalizeHeading(pr.TrueHeading);
198     var navStatus = AisValidator.NormalizeNavStatus(pr.NavigationalStatus);
199     var rateOfTurn = AisValidator.NormalizeRateOfTurn(pr.RateOfTurn);
200
201     await repo.UpsertVesselAsync(new VesselRecord
202     {
203         Mmsi = pr.UserID,
204         TimestampUtc = ts
205     });
206
207     await repo.InsertPositionAsync(new PositionRecord { ... });
```

WHY THE VESSEL IS UPSERTED FIRST — AND IT IS NOT OPTIONAL

`positions` has a foreign key to `vessels`. A position row for an unknown MMSI is rejected by the database.

And position reports nearly always arrive *before* a vessel's first static message — every few seconds versus every few minutes. So for most ships, the very first thing you ever hear is a position for a vessel you have never seen.

The fix is line 201: always upsert the vessel first, using only the MMSI, creating a near-empty row if necessary. The name and IMO get filled in later when static data arrives. This is why the `MERGE` in Chapter 17 uses `COALESCE` so carefully — this call passes `null` for every static field, and must not erase what a previous static message established.

16.5 • HandleStaticData and BuildEta

```
229     int? imo = AisValidator.IsValidImo(sd.ImoNumber) ? sd.ImoNumber : null;
...
241     DimToBow = (short?)sd.Dimension?.A,
245     Draught = sd.MaximumStaticDraught > 0 ? (decimal)sd.MaximumStaticDraught : null,
246     Destination = string.IsNullOrEmpty(sd.Destination) ? null : sd.Destination.Trim(),
247     Eta = BuildEta(sd.Eta),
248     TimestampUtc = DateTime.UtcNow
```

line 241 – (short?)sd.Dimension?.A

Two conversions in one expression. `?.` yields null if `Dimension` is absent; the cast narrows `int` to `short` to match the `SMALLINT` column. Reading right to left is the way to parse this.

line 245 – draught of zero

Zero means "not reported", not "floating on the surface". Converting it to null is the same sentinel-to-null discipline as the validator, applied inline. Arguably it belongs in `AisValidator` with the rest — Chapter 23 returns to this.

line 246 – trimming the destination

AIS pads text fields to a fixed width with spaces, because frames are fixed-size. Without the `Trim()` you would store `"ROTTERDAM "` and every exact-match query would fail. Whitespace-only becomes null rather than an empty string — one representation of "nothing", not two.

line 248 – `DateTime.UtcNow`, not the message timestamp

A subtle inconsistency. Position reports use the timestamp *from the feed*; static messages use *our clock*. So `last_seen_utc` means slightly different things depending on which message updated it last. Worth knowing before you rely on that column for anything precise.

The ETA problem

```
281 private static DateTime? BuildEta(Eta? eta)
282 {
283     if (eta is null) return null;
284     if (eta.Month is < 1 or > 12 || eta.Day is < 1 or > 31) return null;
285     if (eta.Hour > 23 || eta.Minute > 59) return null;
286
287     var year = DateTime.UtcNow.Year;
288     try
289     {
290         return new DateTime(year, eta.Month, eta.Day, eta.Hour, eta.Minute, 0,
291                             DateTimeKind.Utc);
292     }
293     catch (ArgumentOutOfRangeException)
294     {
295         return null;
296     }
```

AIS transmits an ETA as month, day, hour, minute — **and no year**, because the frame has no room for one. So the code has to invent one.

line 284 – the "or" pattern

`eta.Month is < 1 or > 12` is C# 9 pattern matching, equivalent to `eta.Month < 1 || eta.Month > 12` but stating the subject once. Reads almost as English.

line 284 – why day 31 is allowed for every month

Because this check does not know the month yet. 31 February passes the range check and then fails inside the `DateTime` constructor — which is exactly what the try/catch on lines 288-295 is for.

line 287 – assuming the current year

Reasonable, and wrong twice a year in a specific way: on 28 December a ship reporting an ETA of 3 January gets 3 January of the year *just ending* — an ETA nearly twelve months in the past. See Chapter 23.

16.6 · The summary loop and the counters

```
261     while (!stoppingToken.IsCancellationRequested)
262     {
263         await Task.Delay(interval, stoppingToken);
264
265         _logger.LogInformation(
266             "SUMMARYyyyyyyyyy!!!!!!! (last {Seconds}s): received={Received} stored={Stored}
...",
267             _ingestion.SummaryIntervalSeconds,
268             Interlocked.Exchange(ref _received, 0),
269             Interlocked.Exchange(ref _stored, 0),
270             Interlocked.Exchange(ref _rejected, 0),
271             Interlocked.Exchange(ref _staticStored, 0),
272             Interlocked.Exchange(ref _reconnects, 0));
273     }
```

CONCEPT — WHAT INTERLOCKED.EXCHANGE BUYS YOU

Reading a counter and resetting it is two operations:

```
var count = _received;    // read
_received = 0;           // reset
```

If a message arrives *between* those two lines, its increment is destroyed by the reset and that message is never counted. On a firehose that window is hit constantly.

`Interlocked.Exchange(ref _received, 0)` does both in one indivisible CPU instruction: it sets the value to 0 and returns what it was. Nothing can slip in between, because there is no "between".

...BUT THE INCREMENTS ARE NOT PROTECTED

The resets are atomic. The increments are not:

```
_received++;    // read, add one, write back - three steps, interruptible
```

So the exact race the `Exchange` was chosen to prevent can still occur from the other side: the ingestion thread reads `_received` as 500, the summary thread exchanges it to 0 and reports 500, and the ingestion thread then writes back 501 — resurrecting a count that was already reported.

The consistent fix is `Interlocked.Increment(ref _received)` on every counter. In practice the drift is tiny and these are log statistics, not billing, so the impact is low — but the asymmetry is worth seeing, because it shows that a single `Interlocked` call proves nothing unless *every* access to that field is equally protected.

The fire-and-forget question

Return to line 43: `_ = RunSummaryLoopAsync(stoppingToken);` .

Because the `Task` is discarded, nobody ever observes it. If the method threw, the exception would vanish silently and the summaries would simply stop, with no error anywhere — the classic fire-and-forget failure.

The author clearly knew this, because the method wraps its entire body in try/catch and logs on the way out (lines 275-279). That is the right mitigation. Note the ordering, though: `catch (OperationCanceledException) { }` comes first so that shutdown stays silent, and only genuinely unexpected exceptions reach the error log.

AND THE LOG LINE ITSELF

Line 266 reads `"SUMMARYyyyyyyyyy!!!!!!!!!"`. It is debugging residue — presumably from making the line easy to spot in a scrolling console. It should be `"SUMMARY"` before this ships anywhere real. Log output is a user interface, and this one currently addresses an audience of one.

CHECK YOURSELF

1. The feed accepts your connection then immediately closes it, every time. Trace lines 46-66 and say what the reconnect interval actually is.
2. Why must `UpsertVesselAsync` be awaited *before* `InsertPositionAsync`, rather than the two being started together?
3. A ship's static message arrives with `Destination: "ROTTERDAM "`. What is stored, and what would break without line 246?
4. You move `using var ms = new MemoryStream();` from inside the loop (line 109) to just above it. Describe the first symptom you would see.
5. Explain why `_received++` paired with `Interlocked.Exchange` is an inconsistent choice, and what the consistent version is.

17 • Repositories/ — talking to SQL Server

Every line of SQL in the application lives in this one file. No Entity Framework, no Dapper — raw `Microsoft.Data.SqlClient`. That is more code to write, and in exchange you can see exactly what runs against the database, which for a high-write ingestion path is a defensible trade.

The shape every method shares

```
public async Task InsertDeadLetterAsync(string rawPayload, string reason)
{
    await using var conn = new SqlConnection(_connectionString); // 1. connection
    await conn.OpenAsync(); // 2. open it

    const string sql = @"INSERT INTO ..."; // 3. the SQL

    await using var cmd = new SqlCommand(sql, conn); // 4. command
    cmd.Parameters.Add("@RawPayload", SqlDbType.NVarChar, -1)
        .Value = rawPayload; // 5. parameters

    await cmd.ExecuteNonQueryAsync(); // 6. run it
}
```

`const string sql = @"..."`

The `@` prefix makes a **verbatim string literal** — newlines are kept, backslashes are not escapes. It lets multi-line SQL be written readably. `const` is a quiet but important signal: this string is fixed at compile time and therefore *cannot* contain user input.

`await using`

Disposes asynchronously at the end of the method. For a `SqlConnection`, disposal returns it to the pool rather than closing a socket. Chapter 5 covered why `await using` rather than plain `using`.

`ExecuteNonQueryAsync`

One of three execute methods, and choosing correctly matters:

Method	Returns	Use for
<code>ExecuteNonQueryAsync</code>	rows affected	INSERT, UPDATE, DELETE, MERGE
<code>ExecuteScalarAsync</code>	first column of first row	a single value — <code>GetMmsiByImoAsync</code>
<code>ExecuteReaderAsync</code>	a forward-only cursor	rows with several columns

CONCEPT — A NEW CONNECTION PER METHOD IS CORRECT, NOT WASTEFUL

Opening a database connection is genuinely expensive: a TCP handshake, a TLS handshake, and authentication. Doing that thousands of times a minute would be ruinous — so ADO.NET does not do it.

`new SqlConnection(...)` takes a connection from a **pool** that is already open and authenticated. Disposing it returns it to the pool rather than closing it. The pool is keyed on the exact connection string, which is why every method here uses the same `_connectionString` field.

So this pattern costs almost nothing and buys real safety: no connection is ever shared between two operations, and none can be left open by a forgotten code path. **Sharing one long-lived `SqlConnection` across an application is the actual mistake** — it is not thread-safe.

SQL injection, and why this code is immune

The single most exploited web vulnerability of the last twenty-five years, in one comparison:

```
// CATASTROPHIC - never write this
var sql = "SELECT * FROM vessels WHERE mmsi = " + userInput;

// If userInput is: 0; DROP TABLE vessels; --
// the database receives and executes exactly that.

// SAFE - what this project does, everywhere
const string sql = "SELECT mmsi FROM vessels WHERE imo = @Imo;";
cmd.Parameters.Add("@Imo", SqlDbType.Int).Value = imo;
```

The distinction is not "escaping is applied". It is that **the SQL and the data travel separately**. The query text is sent once and planned; the parameter arrives as a typed value in a different part of the protocol. There is no parsing step in which data could become code. A parameter containing `DROP TABLE` is simply an integer conversion that fails, or a string that matches nothing.

Note also the explicit `SqlDbType` on every parameter. Letting the driver infer types works, but stating them prevents surprises — an implicit conversion in a WHERE clause can silently prevent SQL Server from using an index, turning a millisecond lookup into a full table scan.

The MERGE — the most interesting SQL in the project

InfraAis/Repositories/AisRepository.cs — lines 23-50, abridged

```
MERGE vessels WITH (HOLDLOCK) AS target
USING (SELECT @Mmsi AS mmsi) AS source
ON (target.mmsi = source.mmsi)
WHEN MATCHED THEN
    UPDATE SET
        imo           = COALESCE(@Imo, target.imo),
        name          = COALESCE(@Name, target.name),
        call_sign     = COALESCE(@CallSign, target.call_sign),
        ...
        last_seen_utc = @TimestampUtc,
        updated_utc   = SYSUTCDATETIME()
WHEN NOT MATCHED THEN
```

```
INSERT (mmsi, imo, name, ..., first_seen_utc, last_seen_utc, updated_utc)
VALUES (@Mmsi, @Imo, @Name, ..., @TimestampUtc, @TimestampUtc, SYSUTCDATETIME());
```

An **upsert**: insert if the vessel is new, update if it is known. One statement, one round trip, one lock.

Why `COALESCE` is load-bearing

Recall from §16.4 that a position report upserts the vessel with **only** an MMSI — every static field is null. Without `COALESCE`, that update would say `name = NULL` and wipe out the name a static message established minutes earlier. Every position report would erase the ship's identity, and since positions outnumber static messages roughly a hundred to one, your `vessels` table would be permanently empty of names.

`COALESCE(@Name, target.name)` reads: "use the new name, but if it is null, keep the existing one."

ANALOGY — FILLING IN A FORM IN PENCIL

A clerk holds one card per ship. Position reports are notes saying only "vessel 255805678 was here at 09:21". Static messages are notes saying "vessel 255805678 is called BUXCLIFF, IMO 9074729".

`COALESCE` is the instruction: **only overwrite a box on the card if the new note actually has something for that box**. A blank on the note means "no information", not "erase what you have". A clerk who took blanks literally would rub out the ship's name a hundred times an hour.

Why `WITH (HOLDLOCK)`

THE RACE THIS PREVENTS

`MERGE` looks atomic. Without an appropriate lock hint, it is not. Under concurrency this interleaving is possible:

```
Session A: does the row exist? → no
Session B: does the row exist? → no
Session A: INSERT mmsi 255805678 → succeeds
Session B: INSERT mmsi 255805678 → PRIMARY KEY violation
```

`HOLDLOCK` (equivalent to the `SERIALIZABLE` hint) holds a range lock on the key being tested until the statement completes, so B waits for A to finish and then correctly takes the `MATCHED` branch. This is a well-known SQL Server footgun; the code gets it right.

Whether this application is concurrent enough to need it today is a separate question — with one worker writing sequentially, it may not be. It costs little and it makes the statement correct under a future in which two instances run at once, which as Chapter 2 noted is a plausible future.

`USING (SELECT @Mmsi AS mmsi) AS source`

`MERGE` is designed to merge one table into another. Here there is no source table, so a single-row result is manufactured from a parameter to stand in for one.

`last_seen_utc = @TimestampUtc, without COALESCE`

Deliberate. Every message, of either kind, should push this forward — that is the field's whole purpose. By contrast `first_seen_utc` appears only in the `INSERT` branch, so it is written once and never touched again.

SYSUTCDATETIME()

The *database server's* UTC clock, used for `updated_utc`. Distinguish the three timestamps on a vessel row: `msg_timestamp_utc` is when the ship spoke, `received_utc` is when we heard it, and `updated_utc` is when we wrote it down.

The decimal parameter, which needs special handling

```
63     var draught = cmd.Parameters.Add("@Draught", SqlDbType.Decimal);
64     draught.Precision = 4;
65     draught.Scale = 1;
66     draught.Value = (object?)vessel.Draught ?? DBNull.Value;
```

Alone among the parameters, decimal needs its precision and scale stated. The column is `DECIMAL(4,1)` — four significant digits, one after the point, so up to 999.9. Omit these and the driver guesses, typically `DECIMAL(18,0)`, which would silently round a draught of 11.5 metres to 12.

CONCEPT — NULL IS NOT NULL

`(object?)vessel.Draught ?? DBNull.Value` appears on nearly every parameter, and it is not ceremony.

C#'s `null` means "no reference". SQL's `NULL` is a value meaning "unknown", represented in ADO.NET by the singleton `DBNull.Value`. Assign C# `null` to `.Value` and the driver treats the parameter as *unset* — it is not sent at all, and SQL Server rejects the command for a missing parameter.

The cast to `object?` is needed because `??` requires both sides to share a type, and `decimal?` and `DBNull` have nothing in common but `object`.

Reading rows back

```
148     await using var reader = await cmd.ExecuteReaderAsync();
149     if (!await reader.ReadAsync())
150         return null;
151
152     return new LatestPositionRecord
153     {
154         Mmsi = reader.GetInt64(reader.GetOrdinal("mmsi")),
155         Latitude = reader.GetDouble(reader.GetOrdinal("latitude")),
156         Sog = reader.IsDBNull(reader.GetOrdinal("sog"))
157             ? null : reader.GetDecimal(reader.GetOrdinal("sog")),
158         NavStatus = reader.IsDBNull(reader.GetOrdinal("nav_status"))
159             ? null : reader.GetByte(reader.GetOrdinal("nav_status")),
```

line 149 – ReadAsync as the existence test

`ReadAsync` advances to the next row and returns false when there are none. So "no rows" and "vessel not found" are the same condition, and the method returns `null` — which the controller turns into a 404.

GetOrdinal("mmsi")

Column name to index. Slower than hard-coding `0`, `1`, `2`, and enormously more robust: reorder the SELECT list and index-based code silently reads the wrong column, while name-based code keeps working. Correctness

over micro-optimisation, and the right call.

IsDBNull before Get...

Mandatory. `GetDecimal` on a NULL column throws `SqlNullValueException`. Every nullable column must be tested first — which is why the nullable columns are so much noisier than `mmsi` and `latitude`.

line 159 – GetByte for nav_status

The getter must match the SQL type exactly. `nav_status` is `TINYINT`, whose CLR type is `byte` — so `GetByte`. Calling `GetInt32` here throws `InvalidCastException`, not a silent conversion. Consult the mapping when a getter throws.

SQL type	CLR type	Reader method
BIGINT	long	GetInt64
INT	int	GetInt32
SMALLINT	short	GetInt16
TINYINT	byte	GetByte
FLOAT	double	GetDouble
DECIMAL(p,s)	decimal	GetDecimal
DATETIME2	DateTime	GetDateTime
BIT	bool	GetBoolean

The "latest position" query

```
SELECT TOP (1)
    p.mmsi, p.latitude, p.longitude, p.sog, p.cog, p.nav_status, p.msg_timestamp_utc,
    v.imo, v.name
FROM positions p
JOIN vessels v ON v.mmsi = p.mmsi
WHERE p.mmsi = @Mmsi
ORDER BY p.msg_timestamp_utc DESC;
```

Filter to one vessel, sort newest first, take one. The `JOIN` pulls the name and IMO from `vessels` so the API can return "BUXCLIFF" rather than a bare number.

This is the query that makes the covering index in Chapter 21 worth having, and — note — the query that a single future-dated row would poison, which is the concrete reason `IsNotFutureSkewed` exists.

CHECK YOURSELF

1. Someone "simplifies" `COALESCE(@Name, target.name)` to `@Name`. Describe what the `vessels` table looks like an hour later, and why.
2. Why must `IsDBNull` be called before `GetDecimal`, but not before `GetInt64` on `mmsi`?
3. Lines 64-65 set `Precision` and `Scale`. What is stored for a draught of 11.5 metres if you delete them?
4. Explain, in terms of how the query reaches the server, why parameterization stops SQL injection — without using the word "escape".

This class exists to answer one question: **the caller gave us a number — what is it, and what MMSI does it correspond to?** Two methods, and the split between them is the lesson.

Parse — pure, synchronous, no database

InfraAis/Services/VesselIdentifierResolver.cs — lines 22-48

```

22 public VesselIdentifier? Parse(string rawIdentifier, string? idTypeOverride)
23 {
24     if (!long.TryParse(rawIdentifier, out var value) || value <= 0)
25         return null;
26
27     var digits = rawIdentifier.Trim().Length;
28
29     if (idTypeOverride is not null)
30     {
31         switch (idTypeOverride.ToLowerInvariant())
32         {
33             case "mmsi":
34                 return digits == 9 ? new VesselIdentifier { Type = IdentifierType.Mmsi, Value =
value } : null;
35             case "imo":
36                 return digits == 7 ? new VesselIdentifier { Type = IdentifierType.Imo, Value =
value } : null;
37             default:
38                 return null;
39         }
40     }
41
42     return digits switch
43     {
44         9 => new VesselIdentifier { Type = IdentifierType.Mmsi, Value = value },
45         7 => new VesselIdentifier { Type = IdentifierType.Imo, Value = value },
46         _ => null
47     };
48 }

```

line 24 – TryParse, and the guard against zero

Rejects anything non-numeric. The `value <= 0` also rejects `"-9074729"`, which would otherwise parse fine as a negative number.

line 27 – counting characters, not digits of the number

A subtle and correct choice. `"0009074729"` has ten characters but parses to a seven-digit value. Using the string length treats it as malformed, which is right: an identifier is a fixed-width code, not an arithmetic quantity. **This is the whole reason the method takes a `string` rather than a `long`.**

line 31 – ToLowerInvariant

So `"IMO"`, `"imo"`, and `"Imo"` all work. *Invariant* rather than culture-aware matters here: in Turkish, uppercase `I` lowercases to a dotless `ı`, and `"IMO"` would not match `"imo"` on a Turkish-locale server. This is the famous "Turkish I" bug, and `ToLowerInvariant` is the standing fix.

lines 33-36 – the override must agree

If the caller says `?idType=imo` but sends nine digits, the answer is `null` — a 400. It does not silently trust the override, nor silently ignore it. **Contradiction is an error, not something to resolve.**

lines 42-47 – inference by length

A `switch` expression: nine digits means MMSI, seven means IMO, anything else is rejected. This works only because the two identifier formats happen not to overlap in length — a fact worth a comment in the code, since it is load-bearing.

ResolveToMmsiAsync — impure, asynchronous, hits the database

```
50 public async Task<long?> ResolveToMmsiAsync(VesselIdentifier identifier)
51 {
52     if (identifier.Type == IdentifierType.Mmsi)
53         return identifier.Value;
54
55     return await _repo.GetMmsiByImoAsync((int)identifier.Value);
56 }
```

An MMSI is already an MMSI, so it is returned immediately with no database call — and the test at `VesselIdentifierResolverTests` line 83 asserts precisely that, using `mock.Verify(..., Times.Never)`. An IMO requires a lookup, which may find nothing, which is why the return type is `long?`.

WHY SPLIT PARSING FROM RESOLVING?

The two halves have completely different characters, and separating them buys three things.

Failure meaning. `Parse` returning null means "your input is malformed" — a 400.

`ResolveToMmsiAsync` returning null means "your input is fine but we have never seen this ship" — a 404. Two different HTTP answers, so two different methods.

Cost. Malformed input is rejected without touching the database at all.

Testability. `Parse` is pure, so the sixteen resolver tests need no database — and the seven that target `Parse` need not even set up a mock.

CHECK YOURSELF

1. `GET /api/vessels/0009074729` — trace `Parse` and give the HTTP status. Why is that the right answer rather than treating it as IMO 9074729?
2. Why does `Parse` take a `string` instead of a `long`? Give the specific input that justifies it.
3. A request has nine digits and `?idType=imo`. What comes back, and what is the design principle behind that choice?

19 • Controllers/VesselsController.cs — the API

The whole read side. Two endpoints, no business logic — it orchestrates the resolver and the repository and translates their answers into HTTP.

The attributes

```
9 [ApiController]
10 [Route("api/vessels")]
11 public class VesselsController : ControllerBase
```

[ApiController]

Opts into a set of API-specific conveniences: parameters are bound from route and query without annotation, and validation failures become automatic 400 responses. Its presence is also what `AddControllers()` looks for when scanning the assembly.

[Route("api/vessels")]

The prefix for every action in the class.

: ControllerBase

Not `Controller`. `ControllerBase` has everything an API needs — `Ok()`, `NotFound()`, `BadRequest()` — without the view-rendering machinery an HTML site would want. The right base class for JSON.

Routing and binding

```
27 [HttpGet("{identifier}/positions/latest")]
28 public async Task<ActionResult<LatestPositionResponse>> GetLatestPosition(
29     string identifier, [FromQuery] string? idType)
```

Combined with the class-level route, this handles:

```
GET /api/vessels/255805678/positions/latest
GET /api/vessels/9074729/positions/latest?idType=imo
```

{identifier}

A route parameter. The segment in that position is bound to the `identifier` argument by name — the match is on the name, which is why they must agree.

[FromQuery] string? idType

Bound from the query string. Nullable, so it is optional; omit it and the resolver infers the type from the length.

ActionResult<LatestPositionResponse>

A union type meaning "either that object, or an HTTP result". It lets one method return `Ok(response)`, `NotFound(...)`, or `BadRequest(...)` while still telling Swagger what the success shape is.

The four exits, and why they are four

```
31     var parsed = _resolver.Parse(identifier, idType);
32     if (parsed is null)
35         return BadRequest(new { error = "Identifier must be a 7-digit IMO or 9-digit MMSI;
idType must match." });
38     var mmsi = await _resolver.ResolveToMmsiAsync(parsed);
39     if (mmsi is null)
42         return NotFound(new { error = "No vessel known with this IMO." });
45     var latest = await _repo.GetLatestPositionAsync(mmsi.Value);
46     if (latest is null)
47     {
48         if (await _repo.GetVesselAsync(mmsi.Value) is null)
51             return NotFound(new { error = "No vessel known with this MMSI." });
54         return NotFound(new { error = "Vessel is known but has no stored positions yet." });
55     }
58     return Ok(new LatestPositionResponse { ... });
```

Status	Condition	What the caller should conclude
400	Not 7 or 9 digits, or the override contradicts the length	"Your request is wrong." Fix the request.
404	An IMO we have no vessel for	"Valid request, unknown ship." We have never received static data linking that IMO.
404	An MMSI we have no vessel for	"Valid request, unknown ship." Never heard from it at all.
404	Vessel exists, no positions stored	"We know the ship, we have no fix." Keep polling — this one may resolve itself.
200	Found	The position, plus <code>ageSeconds</code> .

WHY THE LAST TWO ARE WORTH THE EXTRA QUERY

Line 48 costs an additional database round trip purely to distinguish two 404s that a lazier implementation would merge into one message.

That extra query buys something real. "We have never heard of this ship" tells an integrator their identifier is wrong or the vessel is outside the bounding box. "We know this ship but have no position" tells them the identifier is right and to try again shortly. **Same status code, completely different next action** — and the difference is invisible to the caller unless the server distinguishes it.

Two of the twelve controller tests exist solely to hold this behaviour in place, including one (`GetLatest_TwoNotFoundCases_HaveDifferentMessages`) whose only assertion is that the two messages are not equal. That is a test protecting a deliberate design decision from a future well-meaning simplification.

The mapping at the end

```
68     NavStatusText = NavStatusMapper.ToText(latest.NavStatus),
70     AgeSeconds = Math.Round((DateTime.UtcNow - latest.MsgTimestampUtc).TotalSeconds)
```

The two computed fields from Chapter 14. Subtracting two `DateTime` s gives a `TimeSpan` ; `.TotalSeconds` is the whole span expressed in seconds as a `double` , not the seconds component. `Math.Round` then drops a meaningless fractional part.

This is the only place in the read path that depends on the current time — worth remembering, because it is also the only thing making these responses non-deterministic, and it is why the success test asserts `InRange(body.AgeSeconds, 85, 95)` rather than an exact value.

CHECK YOURSELF

1. Why is a malformed identifier a 400 while an unknown one is a 404? State the rule in one sentence.
2. What is the practical value to an API consumer of separating the two 404 messages, given that both are 404?
3. The controller contains no SQL and no validation rules. Which classes hold each, and why is that separation worth the extra files?

InfraAis/Utils/NavStatusMapper.cs — complete

```
public static class NavStatusMapper
{
    public static string ToText(int? navStatus) => navStatus switch
    {
        0 => "Under way using engine",
        1 => "At anchor",
        2 => "Not under command",
        3 => "Restricted manoeuvrability",
        4 => "Constrained by her draught",
        5 => "Moored",
        6 => "Aground",
        7 => "Engaged in fishing",
        8 => "Under way sailing",
        14 => "AIS-SART active",
        15 => "Not defined",
        null => "Unknown",
        _ => "Reserved"
    };
}
```

Twenty lines, and it does something disproportionately valuable: it turns a number that means nothing to a human into a phrase that does.

`=> on the method`

An expression-bodied member. The method is a single expression, so no braces and no `return`.

`navStatus switch { ... }`

A **switch expression** (C# 8), not a switch statement. It produces a value rather than executing branches — which is why it can be the entire body.

`null => "Unknown"`

Null is a matchable pattern, so the "we never received this" case is handled in the same table as the real codes rather than by a separate `if` above it.

`_ => "Reserved"`

The discard pattern — the catch-all. Codes 9 through 13 are reserved by the AIS specification for future use, so anything unlisted gets an honest label instead of a crash. **The switch expression is exhaustive by construction:** leave out the `_` and an unmatched value throws at runtime, so the compiler pushes you towards completeness.

Note where this sits: `Utils`, static, no dependencies, no state. It is called from the controller during response mapping and never from the ingestion path — the database stores the code, and the text is derived on read. Storing the text instead would duplicate a 25-character string across millions of rows and freeze today's wording into historical data.

CHECK YOURSELF

1. Why store `nav_status = 0` rather than "Under way using engine"? Give two reasons.
2. What happens if the AIS standard assigns a meaning to code 11 and you have not updated this file?
3. You delete the `_ => "Reserved"` line. What happens when code 11 arrives — and at compile time or run time?

21 · The database schema

Three tables. The design decisions in them are worth as much attention as the C#.

vessels — one row per ship, updated in place

```
CREATE TABLE vessels (  
    mmsi          BIGINT          NOT NULL PRIMARY KEY,  
    imo           INT             NULL,  
    name          NVARCHAR(128)  NULL,  
    call_sign     NVARCHAR(16)   NULL,  
    ship_type     SMALLINT        NULL,  
    dim_to_bow    SMALLINT        NULL,  
    ...  
    draught       DECIMAL(4,1)   NULL,  
    destination   NVARCHAR(64)   NULL,  
    eta           DATETIME2       NULL,  
    first_seen_utc DATETIME2       NOT NULL,  
    last_seen_utc DATETIME2       NOT NULL,  
    updated_utc   DATETIME2       NOT NULL  
    CONSTRAINT DF_vessels_updated DEFAULT SYSUTCDATETIME()  
);  
  
CREATE UNIQUE INDEX UX_vessels_imo ON vessels(imo) WHERE imo IS NOT NULL;
```

mmsi BIGINT PRIMARY KEY

A natural key — the real-world identifier is the primary key, with no surrogate `id`. Defensible here because MMSI is unique by international allocation and never null. `BIGINT` rather than `INT` because a 9-digit MMSI can reach 999,999,999, uncomfortably close to `INT`'s ceiling of 2,147,483,647.

NVARCHAR, not VARCHAR

`N` means Unicode. Ship names contain Norwegian *ø*, German *ü*, Greek and Cyrillic letters. `VARCHAR` would mangle them.

DECIMAL(4,1) for draught

Up to 999.9 metres with one decimal place. The deepest ships draw about 25 metres, so this is comfortable, and — per Chapter 17 — it is the reason the parameter needs explicit precision and scale.

the filtered unique index

The cleverest line in the schema. A plain unique index would allow only one row with a NULL `imo`, because standard SQL treats NULLs as equal for uniqueness. Since most vessels have no IMO yet, that would be catastrophic. The `WHERE imo IS NOT NULL` clause indexes only rows that have an IMO — so IMOs are unique, unlimited rows may have none, and `GetMmsiByImoAsync` gets an index seek for free.

positions — append-only, grows forever

```
CREATE TABLE positions (  
    id                BIGINT          IDENTITY(1,1) PRIMARY KEY,  
    mmsi              BIGINT          NOT NULL,  
    latitude          FLOAT           NOT NULL,  
    longitude         FLOAT           NOT NULL,  
    sog               DECIMAL(5,1)    NULL,  
    ...  
    msg_timestamp_utc DATETIME2       NOT NULL,  
    received_utc      DATETIME2       NOT NULL  
    CONSTRAINT DF_positions_received DEFAULT SYSUTCDATETIME(),  
    CONSTRAINT FK_positions_vessels FOREIGN KEY (mmsi) REFERENCES vessels(mmsi)  
);  
  
CREATE INDEX IX_positions_mmsi_time  
    ON positions(mmsi, msg_timestamp_utc DESC)  
    INCLUDE (latitude, longitude, sog, cog, nav_status);
```

id IDENTITY(1,1)

A surrogate key here, unlike `vessels` — because no natural key exists. The same ship can legitimately report twice in the same second, so `(mmsi, timestamp)` is not reliably unique. This is exactly why deduplication was left out; see the README's design notes.

two timestamps

`msg_timestamp_utc` is when the ship spoke; `received_utc` is when we wrote it down. Keeping both lets you measure end-to-end pipeline lag, and diagnose a backlog after an outage.

the foreign key

The constraint that forces the upsert-before-insert ordering of §16.4. It could have been dropped for write throughput; keeping it means the database itself guarantees no orphan positions.

the covering index

Read it against the query in Chapter 17. Key columns `(mmsi, msg_timestamp_utc DESC)` match the WHERE and the ORDER BY exactly, so SQL Server seeks straight to the newest row for that vessel with no sort. The `INCLUDE` columns then carry the payload *inside the index*, so the answer is produced without touching the table at all. **That is what "covering" means, and it is the difference between a constant-time lookup and a scan over millions of rows.**

ingestion_dead_letter

```
CREATE TABLE ingestion_dead_letter (  
    id                BIGINT          IDENTITY(1,1) PRIMARY KEY,  
    raw_payload       NVARCHAR(MAX)   NOT NULL,  
    reason            NVARCHAR(256)   NOT NULL,  
    received_utc      DATETIME2       NOT NULL  
    CONSTRAINT DF_dlq_received DEFAULT SYSUTCDATETIME()  
);
```

The audit trail for everything rejected. `NVARCHAR(MAX)` stores the payload whole, whatever its size. The pattern is borrowed from message queues, where a "dead letter queue" holds messages that could not be

processed, and it converts an invisible failure into a queryable one:

```
SELECT reason, COUNT(*) FROM ingestion_dead_letter  
GROUP BY reason ORDER BY COUNT(*) DESC;
```

Run that after an hour and you learn whether your feed has a clock problem, a coverage problem, or a validation rule that is too strict.

CHECK YOURSELF

1. Why does `UX_vessels_imo` need `WHERE imo IS NOT NULL` ? What breaks on the second vessel without it?
2. `vessels` uses a natural primary key and `positions` uses a surrogate one. Justify each.
3. What does `INCLUDE (latitude, longitude, ...)` add that the key columns do not, and which query does it serve?

22 · The tests

Eighty tests, all passing, running in well under a second because not one of them touches a database or a network.

File	Tests	Covers
<code>AisValidatorTests.cs</code>	41	MMSI, position, sentinels, timestamps, skew, nav status, rate of turn
<code>AisValidatorImoTests.cs</code>	11	The IMO checksum in isolation
<code>VesselIdentifierResolverTests.cs</code>	16	Parsing, overrides, IMO→MMSI resolution
<code>VesselsControllerTests.cs</code>	12	All five HTTP outcomes, plus the nav-status mapping

[Fact] versus [Theory]

```
// [Fact] - one test, no inputs
[Fact]
public void NormalizeSog_ReturnsNull_ForSentinel()
{
    Assert.Null(AisValidator.NormalizeSog(102.3));
}

// [Theory] - the same test run once per data row. This is FIVE tests.
[Theory]
[InlineData(91, 181)]
[InlineData(91, 3.1)]
[InlineData(51.4, 181)]
[InlineData(-91, 0)]
[InlineData(0, -181)]
public void IsValidPosition_ReturnsFalse_ForOutOfRange(double lat, double lon)
{
    Assert.False(AisValidator.IsValidPosition(lat, lon));
}
```

Each `InlineData` row is reported separately, so a failure names the exact input that broke. This is why 80 tests come from far fewer methods.

The naming convention

```
IsValidPosition_ReturnsFalse_ForOutOfRange
  ↑ method      ↑ expected      ↑ condition
```

Method_ExpectedResult_Condition. Its value shows up in the failure report: you learn what broke without opening the file. Adopt it.

Mocking, and what the controller tests really assert

```
var (ctrl, resolver, repo) = Build();
resolver.Setup(r => r.Parse("255805678", null)).Returns(parsed);
resolver.Setup(r => r.ResolveToMmsiAsync(parsed)).ReturnsAsync(255805678);
repo.Setup(r => r.GetLatestPositionAsync(255805678)).ReturnsAsync((LatestPositionRecord?)null);
repo.Setup(r => r.GetVesselAsync(255805678)).ReturnsAsync(new VesselRecord { Mmsi = 255805678 });

var result = await ctrl.GetLatestPosition("255805678", null);

var body = Assert.IsType<NotFoundObjectResult>(result.Result).Value!.ToString();
Assert.Contains("no stored positions", body);
```

Setup(...).Returns(...)

"When this exact call is made, return this." Everything not set up returns the type's default, which is usually null — occasionally a trap worth remembering.

ReturnsAsync

The variant for methods returning `Task<T>`. It wraps the value in a completed Task so the `await` in the controller resolves immediately.

the scenario being built

Read the four setups together: a valid MMSI, that resolves, whose latest position is **null**, but whose vessel row **exists**. That is the "known ship, no fix" branch — a state that would be tedious to create in a real database and trivial to create here.

Verify(..., Times.Never)

Used elsewhere to assert a call did *not* happen — for example, that a malformed identifier never reaches the database. Asserting on absence is something mocks make possible and integration tests generally do not.

ARRANGE, ACT, ASSERT

Every test in this project follows the same three-part shape, and the blank lines between them are load-bearing:

Arrange — build the mocks and set up returns. **Act** — call the one method under test, on a single line. **Assert** — check the outcome.

If the Act section needs more than one line, that is usually a signal the test is covering more than one behaviour and wants splitting.

What is not tested, and whether that is acceptable

Nothing covers `AisIngestionService`, `AisRepository`, or the SQL. That is a reasonable Phase 1 position — those need a live WebSocket and a live database — but be clear-eyed about what it means:

Untested area	What would catch it
The <code>MERGE</code> , including the <code>COALESCE</code> behaviour	An integration test against a real or containerised SQL Server
Reconnect and backoff	A test double for the WebSocket, behind an interface that does not yet exist
Frame reassembly across buffer boundaries	The same
The end-to-end message path	An integration test with a stub feed

The pure logic — the part where the subtle bugs live — is thoroughly covered. The I/O is not. That is the normal trade, and it is precisely why `AisValidator` being pure was worth so much: purity is what made the 52 tests possible.

CHECK YOURSELF

1. Why do 80 tests come from roughly 30 methods?
2. Name a bug in the `MERGE` statement that the current suite would not catch, and describe the test that would.
3. `GetLatest_TwoNotFoundCases_HaveDifferentMessages` asserts only that two strings differ. What is it really protecting, and from whom?

Judgment

Reading code is one skill; judging it is another. This part is the critique a senior developer would give at review — specific, ranked, and with a fix for each.

23 · Twelve things in this code worth questioning

First, the fair assessment: **this is good code**. The layering is clean, the SQL is parameterized throughout, the validation is pure and well tested, the dead-letter trail means nothing is silently lost, and several genuinely subtle things — `HOLDLOCK`, the filtered unique index, frame reassembly, the `InvariantCulture` parse — are handled correctly by people who often get them wrong.

What follows is not a list of failures. It is the list of things a reviewer would raise, roughly in order of how much they matter.

■ 1 · A clean close reconnects with no delay at all

Where: `AisIngestionService.cs` lines 50-52 and 114-117.

The backoff protects only the *exception* path. When the server closes the socket politely, `ConnectAndListenAsync` returns normally, line 52 resets the delay, and the loop reconnects immediately.

An invalid API key or a rate limit typically produces exactly that: accept, then close. The result is a reconnect loop running as fast as the network allows — thousands of connections a minute against the provider, which is how you get an IP ban.

```
// Fix: distinguish a session that did real work from one that did not.
var connectedAt = DateTime.UtcNow;
await ConnectAndListenAsync(stoppingToken);

if (DateTime.UtcNow - connectedAt > TimeSpan.FromSeconds(30))
    delay = _ingestion.ReconnectBaseDelaySeconds; // genuinely healthy - reset
else
{
    await Task.Delay(TimeSpan.FromSeconds(delay), stoppingToken);
    delay = Math.Min(delay * 2, _ingestion.ReconnectMaxDelaySeconds);
}
```

■ 2 · Static messages skip MMSI validation

Where: `AisIngestionService.cs` lines 224-249.

Compare the two handlers. `HandlePositionReport` checks `IsValidMmsi(pr.UserID)` and dead-letters the message if it fails. `HandleStaticData` checks the *IMO* — but sends `sd.UserID` straight into `UpsertVesselAsync` with no check at all.

So a malformed static message creates a junk row in `vessels`, keyed by an invalid MMSI, which no position report will ever match and which the API will never return. The table slowly fills with phantom ships. It also bypasses the dead-letter trail entirely — the one thing the design promises never to do.

```
// Fix: apply the same guard as the position path.
if (!AisValidator.IsValidMmsi(sd.UserID))
{
    await repo.InsertDeadLetterAsync(rawPayload, $"bad MMSI {sd.UserID} (static)");
    _rejected++;
    return;
}
```

This is the finding I would raise first in a review, because it is an *asymmetry*. Two paths that should enforce the same rule do not, and nothing in the code explains why.

■ 3 · No configuration validation at start-up

Where: `Program.cs` lines 9-16.

Nothing checks the bound values. Three concrete consequences:

Bad configuration	Symptom
Connection string missing or misspelled	App starts fine; first database write fails several seconds later with a message about "data source" that says nothing about configuration.
<code>SummaryIntervalSeconds: 0</code>	<code>Task.Delay(TimeSpan.Zero)</code> in a loop — a hot loop producing thousands of log lines a second.
<code>ReconnectBaseDelaySeconds: -1</code>	<code>Task.Delay</code> throws <code>ArgumentOutOfRangeException</code> — but only on the first reconnect, possibly days in.

```
// Fix: fail at boot, with a message naming the setting.
builder.Services
    .AddOptions<IngestionOptions>()
    .Bind(builder.Configuration.GetSection(IngestionOptions.SectionName))
    .Validate(o => o.SummaryIntervalSeconds > 0, "SummaryIntervalSeconds must be > 0")
    .Validate(o => o.ReconnectBaseDelaySeconds > 0, "ReconnectBaseDelaySeconds must be > 0")
    .Validate(o => o.ReconnectMaxDelaySeconds >= o.ReconnectBaseDelaySeconds, "max must be >=
base")
    .ValidateOnStart();

builder.Services
    .AddOptions<AisDbOptions>()
    .Bind(builder.Configuration.GetSection(AisDbOptions.SectionName))
    .Validate(o => !string.IsNullOrEmpty(o.AisDb), "ConnectionStrings:AisDb is required")
    .ValidateOnStart();
```

`ValidateOnStart()` is the operative call. Without it validation runs lazily on first access, which for the reconnect settings could be days after deployment. **A configuration error should stop the process at boot, not ambush you at 3 a.m.**

■ 4 · Per-message logging at Information level

Where: lines 221 and 251.

```
_logger.LogInformation("STORED: MMSI={Mmsi} Lat={Lat} Lon={Lon}", ...);
```

The README states that per-message logging is at Debug so the console shows the periodic summary rather than a flood. The code does the opposite. At a realistic global feed rate — hundreds of messages a second — this produces hundreds of log lines a second, which buries the summary line the code went to the trouble of building, and in a hosted environment costs real money in log ingestion.

Fix: `LogDebug` for both. The counters already carry the information at Information level.

■ 5 · The ETA year is wrong for two weeks a year

Where: `BuildEta`, line 287.

AIS sends no year, so the code assumes the current one. On 28 December, a ship reporting an ETA of 3 January is recorded as arriving on **3 January of the year now ending** — an ETA eleven months in the past.

```
// Fix: if the naive date is far in the past, it must mean next year.
var now = DateTime.UtcNow;
var candidate = new DateTime(now.Year, eta.Month, eta.Day, eta.Hour, eta.Minute, 0,
    DateTimeKind.Utc);

if (candidate < now.AddMonths(-6))
    candidate = candidate.AddYears(1);

return candidate;
```

The six-month window is the usual heuristic: voyages are short relative to a year, so an ETA far in the past is almost certainly next year's.

■ 6 · Counters increment without `Interlocked`

Where: lines 124, 220, 250, 60 versus lines 268-272.

Covered in §16.6. The resets are atomic; the increments are not. **Fix:** `Interlocked.Increment(ref _received)` everywhere. Low impact — these are log statistics — but the inconsistency signals a misunderstanding, and a reviewer will read it that way.

■ 7 · Speeds above the sentinel are stored as real

Where: `NormalizeSog`, line 18.

The valid SOG range is 0 to 102.2 knots, with 102.3 meaning "unknown". The check catches exactly 102.3 — so a corrupt value of 500 knots passes through and is stored as a genuine speed.

The related nit is the floating-point equality itself. It works, because the JSON text `102.3` and the literal `102.3` produce identical bits, but it is fragile in a way a range check is not.

```
// Fix: one range check subsumes both problems - as NormalizeCog already does.
public static decimal? NormalizeSog(double sog)
    => (sog < 0 || sog >= 102.3) ? null : (decimal)sog;
```

Note this is precisely the pattern the rest of the class already follows. `NormalizeCog` uses `>= 360`, `NormalizeRateOfTurn` uses a range. `NormalizeSog` is the odd one out.

■ 8 · `last_seen_utc` means two different things

Where: line 204 versus line 248.

Position reports set it from the *message* timestamp; static messages set it from `DateTime.UtcNow` — *our* clock. So the column's meaning depends on which message type last touched the row.

The gap is usually small, but it makes "when did we last hear from this ship?" imprecise, and a query comparing `last_seen_utc` against position timestamps can produce impossible orderings. **Fix:** parse `msg.Metadata.TimeUtc` in the static handler too, exactly as the position handler does.

■ 9 · Two round trips per message, no batching

Where: lines 201 and 207.

Every position report performs an upsert and an insert, each taking a pooled connection. At a few hundred messages a second that is a thousand or more round trips a second, and the ingestion loop `await`s each one — so the reader is blocked while the writer works. If SQL Server slows, messages back up in the socket buffer and are eventually dropped by the provider.

The README already lists the fix as future work, and the shape is standard: a bounded `Channel<PositionRecord>` between the reader and a writer task, with the writer using `SqlBulkCopy` on a

flush interval. Worth knowing it is a known limit and not an oversight.

■ 10 · Sentinel handling leaked into the service

Where: lines 245-246.

```
Draught = sd.MaximumStaticDraught > 0 ? (decimal)sd.MaximumStaticDraught : null,  
Destination = string.IsNullOrEmpty(sd.Destination) ? null : sd.Destination.Trim(),
```

Both are normalization rules — the exact job `AisValidator` exists for. Sitting inline in the service, they are untested (the validator's 52 tests cannot reach them) and invisible to anyone auditing the validation rules in one place.

Fix: `AisValidator.NormalizeDraught(...)` and `AisValidator.NormalizeText(...)`, then test them. The value is not tidiness — it is that the rules become testable.

■ 11 · `AisValidator` should be a static class

Every member is static, but the class is declared `public class AisValidator`. Nothing stops someone writing `new AisValidator()`, which is meaningless. Declaring it `public static class` makes that a compile error and documents the intent. One word.

■ 12 · The log line that should not ship

```
"SUMMARYyyyyyyyyy!!!!!!! (last {Seconds}s): received={Received} ..."
```

Debugging residue at line 266. It also breaks log searching, since nobody greps for that. Change it to `"SUMMARY"`. Trivial to fix, and the kind of thing that shapes a reviewer's impression of everything else.

HOW TO RECEIVE A REVIEW LIKE THIS

Twelve items on one project can feel like a verdict. It is not. Findings 1, 2 and 3 are worth fixing before this runs anywhere real; 4 through 10 are ordinary hardening; 11 and 12 are polish.

More usefully, notice the *shape* of the real findings. Almost all of them are **inconsistencies** rather than ignorance: one path validates and its twin does not; resets are atomic and increments are not; one normalizer uses a range and the others use equality. The author knew every technique involved — the defects are in applying them uniformly. That is what code review is actually for, and it is why reading your own code a week later is worth so much.

Term	Meaning in this project
AIS	Automatic Identification System. The unencrypted radio protocol ships use to broadcast identity and position.
MMSI	9-digit radio identifier. Always present; changes if a ship is reflagged. Primary key of <code>vessels</code> .
IMO number	7-digit permanent hull identifier with a check digit. Appears only in static messages.
SOG / COG	Speed Over Ground (knots) / Course Over Ground (degrees). The direction of travel — not necessarily where the bow points.
True heading	Where the bow actually points. Differs from COG in a current or crosswind.
Sentinel value	A magic number meaning "unknown" — 102.3, 360, 511, 91/181. Normalized to NULL on the way in.
Dead letter	A message that could not be processed, stored raw with a reason instead of being discarded.
DTO	Data Transfer Object. A class whose only job is to carry data across a boundary.
Upsert	Insert if absent, update if present. Done here with <code>MERGE</code> .
Dependency injection	A class declares what it needs in its constructor; the container supplies it.
DI container	The registry of "when asked for X, build Y". Populated in <code>Program.cs</code> .
Composition root	The single place where the object graph is wired — here, <code>Program.cs</code> .
Singleton / Scoped / Transient	Service lifetimes: one per app / one per request / one per injection.
Captive dependency	A short-lived service trapped inside a long-lived one. Avoided here with <code>IServiceScopeFactory</code> .
Hosted service	Code the host starts at boot and runs until shutdown. <code>BackgroundService</code> is the base class.
CancellationToken	A cooperative stop signal. It asks; it cannot force.
Exception filter	The <code>when (...)</code> clause on a <code>catch</code> , used to separate shutdown from failure.
Task	A receipt for work that will complete later. The .NET equivalent of a Promise.
Fire-and-forget	Starting a Task without awaiting it. Exceptions vanish silently.
Options pattern	Binding configuration sections onto typed classes, consumed as <code>IOptions<T></code> .
User secrets	Development-only configuration stored outside the repository, keyed by <code>UserSecretsId</code> .
Nullable reference types	The <code>?</code> annotations. Compile-time warnings, not runtime guarantees.
Sentinel-to-NULL	The normalization discipline: an unknown value becomes a database NULL, never a magic number.
Parameterized query	SQL and data sent separately, so data can never be parsed as code. The defence against injection.

Connection pooling	Reuse of open connections, keyed by connection string. What makes connection-per-method cheap.
MERGE / HOLDLOCK	The upsert statement, with a range lock that prevents a concurrent insert race.
COALESCE	"First non-null wins." Stops a position report erasing static fields.
Filtered index	An index with a <code>WHERE</code> clause. Here it makes IMO unique among rows that have one.
Covering index	An index carrying enough columns to answer a query without touching the table.
Natural / surrogate key	A real-world identifier (<code>mmsi</code>) versus a generated one (<code>id IDENTITY</code>).
DBNull.Value	ADO.NET's representation of SQL NULL. Distinct from C# <code>null</code> .
Exponential backoff	Doubling the retry delay up to a cap, resetting after success.
WebSocket frame	The unit a socket delivers. One message may span several; reassemble until <code>EndOfMessage</code> .
Interlocked	Atomic read-modify-write operations, safe against concurrent access.
xUnit Fact / Theory	A single test / a test run once per <code>InlineData</code> row.
Mock	A fake implementation of an interface, with scripted return values. Moq provides them here.
Arrange–Act–Assert	The three-part shape of every test in this project.
Pure function	Output depends only on input; no I/O, no clock, no state. Why <code>AisValidator</code> is so testable.

25 · Answers to the checks

Compare these against your own answers rather than reading them first. Where yours differs and you can defend it, you have learned more than the answer teaches.

Chapter 1 — What this program does

1. **The MMSI changes; the IMO does not.** The `vessels` table gains a second row — same physical ship, new MMSI, so a new primary key. The old row lingers with a stale `last_seen_utc`. The IMO would identify them as one hull, which is why the unique index on `imo` matters and why a reflag can cause a conflict.
2. Frame size. AIS messages are fixed-width radio bursts with no room for both identifiers, and position reports are sent every few seconds — the MMSI alone is what the radio layer needs.
3. Three ingestion workers would all connect to the feed and write the same messages, tripling the rows in `positions` and triggering `MERGE` contention on `vessels`. The API would work fine; the data would be wrong.

Chapter 3 — Project layout

1. `ImplicitUsings` is enabled, so the compiler adds the common ASP.NET Core namespaces to every file.
2. In the user-profile secrets store, keyed by the `UserSecretsId` GUID — outside the repository, so it cannot be committed.
3. A circular project reference, which MSBuild rejects outright. It would also mean shipping test code and the test framework to production.

Chapter 4 — Nullable reference types

1. `null`. The receiving variable must be nullable — hence the cast to `short?` rather than `short`.
2. Every AIS message carries an MMSI, so it is always known; the IMO appears only in static messages and is discarded when its checksum fails, so it is frequently absent.
3. Because the warning was the only evidence that a null was possible there. `!` deletes the evidence without changing the risk.

Chapter 5 — async/await

1. No. It releases the current thread until the work completes. The waiter does not clone himself — he goes and serves another table.
2. `async` is an instruction to the compiler about how to build the method body. Callers only see the return type, `Task<T>`, so an implementation may be async or not without affecting the contract.
3. The method returns before the insert completes, so you do not know whether it succeeded; and any exception is captured in a Task nobody observes, so failures are silent. Rows go missing with a clean log.

Chapter 6 — Dependency injection

1. At runtime, on the first request to that controller, with `Unable to resolve service for type 'IClock'`. Not at compile time — that is the standing cost of DI.

2. The first is the service being requested (the interface); the second is the implementation to build. "When asked for this, construct that."
3. The class takes its dependencies as constructor parameters and never constructs them itself, so anything satisfying the interfaces will do — mocks included.

Chapter 7 — Lifetimes

1. One HTTP request. ASP.NET Core opens it when the request arrives and disposes it after the response is sent.
2. `Cannot consume scoped service 'InfraAis.Repositories.IAisRepository' from singleton 'InfraAis.Services.AisIngestionService'.`
3. Scopes are never disposed, so every scope and everything created in it stays alive. Memory climbs steadily — at feed rates, thousands of leaked scopes per minute, ending in an out-of-memory crash hours or days later.

Chapter 8 — Options

1. It does not crash. Every value falls back to the property initialisers, so `SummaryIntervalSeconds` is 30.
2. Nothing happens. `IOptions<T>` is read once at start-up; the value was captured in `_ingestion` in the constructor. You would need `IOptionsMonitor<T>`, read at the point of use.
3. The environment variable. It sits later in the configuration chain, and later sources override earlier ones.

Chapter 9 — Interfaces

1. `async` describes how a method body is compiled, not what it promises. An interface declares only the contract — the return type. Implementations choose.
2. All database access goes through `IAisRepository`, so tests substitute a mock. Without the interface the controller would be welded to `AisRepository` and a real connection string.
3. It is pure and stateless — no I/O, nothing to fake, nothing to swap. There is no seam because there is no dependency. Interfaces earn their keep by isolating I/O.

Chapter 10 — BackgroundService

1. If it returns, the service is finished and simply stops — the rest of the app keeps running. If it throws, the host logs it and, by default in .NET 8, stops the application.
2. Ctrl+C appears to hang. The loop cannot notice cancellation until the delay elapses, so shutdown takes up to 30 seconds instead of milliseconds.
3. Without the filter it also swallows cancellations that were *not* caused by shutdown — a network timeout, for instance — and treats a genuine fault as a clean stop, skipping the reconnect.

Chapter 11 — Program.cs

1. Yes. Everything before `builder.Build()` is registration into a dictionary; order is irrelevant.
2. No. After `Build()` you are describing an ordered pipeline, and every request passes through the layers in the order added. Authorization after routing means the request is already handled.
3. Every endpoint still works. The data simply stops getting newer — `ageSeconds` on every response climbs steadily, because nothing is writing to the database any more.

Chapter 12 — Options classes

1. `Url` being null would give an empty string where a string is expected; `BoundingBox` being null would make `_options.BoundingBox.MinLat` throw a `NullReferenceException`. Nested objects need an instance, not just a default value.
2. At the first database operation, seconds after start-up, with an `ArgumentException` about the 'data source' keyword — a message that says nothing about configuration.
3. Renaming the JSON section updates one `const` instead of hunting literals across files — and a typo in the constant is a compile error, whereas a typo in a literal silently binds nothing.

Chapter 13 — AisMessage

1. `TimeUtc` stays `""`, `TryParseTimestamp` returns false, and every position report is dead-lettered as "unparseable timestamp". The log fills with rejections and `SELECT COUNT(*) FROM positions` returns whatever it was an hour ago — no growth at all.
2. Because each individual message contains only one of them. Whichever is absent deserializes to null, so the type must permit it.
3. Nullability here would mean interpreting AIS semantics during deserialization, mixing "what the wire said" with "what it means". The DTO layer should mirror the wire faithfully; interpretation belongs in `AisValidator`.

Chapter 14 — Models

1. Because it changes every second. Storing it would mean rewriting every row continuously; computing it at request time is free and always correct.
2. `Latitude` is a physical measurement stored in a `FLOAT` column, where binary floating point is the right fit. `Sog` maps to `DECIMAL(5,1)`, and matching `decimal` avoids a silent conversion and keeps the value exact.
3. You lose `NavStatusText` and `AgeSeconds` — the two fields a caller most wants. The future bug: renaming a database column silently changes the public JSON contract, or an internal field added to the storage model is suddenly exposed to every consumer.

Chapter 15 — AisValidator

1. `IsValidPosition`, and 91 appears nowhere in it. The plain range check `lat <= 90` rejects it — the sentinel is handled by the general rule.
2. **Invalid.** $9 \times 7 = 63$, $7 \times 6 = 42$, $0 \times 5 = 0$, $4 \times 4 = 16$, $6 \times 3 = 18$, $1 \times 2 = 2 \rightarrow$ sum 141 $\rightarrow$ last digit 1, but the check digit is 2. (9704611 is valid, which is why it appears in the test data.)
3. On a machine whose culture uses a comma as the decimal separator — much of Europe — the fractional seconds fail to parse and every message is dead-lettered. You would not notice on a UK or US developer machine, which is what makes it a classic production-only failure.
4. "Latest position" is `ORDER BY msg_timestamp_utc DESC`. A row dated next Tuesday outranks every genuine report until Tuesday arrives, so the API serves that one stale row for days. A past-dated row simply sorts below the real data and is harmless.

Chapter 16 — AisIngestionService

1. Zero. The clean-close path returns normally, so line 52 resets the delay and the loop reconnects immediately — a tight loop. See finding 1 in Chapter 23.
2. The foreign key. `positions.mmsi` references `vessels.mmsi`, so the parent row must exist and be committed before the child insert; running them concurrently would race and fail.
3. `"ROTTERDAM"` is stored, trimmed. Without line 246 you would store the trailing spaces, and `WHERE destination = 'ROTTERDAM'` would match nothing — a bug that looks like missing data rather than a formatting fault.
4. Every message would be appended to the same stream, so message two would be parsed as message one concatenated with message two — invalid JSON. You would see a `JsonException` on every message after the first, and `positions` would gain exactly one row per connection.
5. `Interlocked.Exchange` makes the reset atomic, but `++` is a non-atomic read-modify-write, so an increment in flight during a reset can resurrect an already-reported count. The consistent version is `Interlocked.Increment(ref _received)` for every counter.

Chapter 17 — Repository

1. Every position report — roughly a hundred times more frequent than static messages — would set `name`, `imo`, and every other static field to NULL. Within minutes the `vessels` table would contain nothing but MMSIs and timestamps.
2. `mmsi` is `NOT NULL`, so it can never be DBNull. `sog` is nullable, and calling `GetDecimal` on a NULL throws `SqlNullValueException`.
3. The driver infers a default such as `DECIMAL(18,0)` — zero decimal places — so 11.5 is rounded to 12. Silently, on every row.
4. The query text and the parameter values travel to the server in different parts of the protocol. The server parses and plans the SQL *before* the values are attached, so there is no stage at which a value could be read as SQL syntax.

Chapter 18 — Resolver

1. `long.TryParse` succeeds with 9074729, but `rawIdentifier.Trim().Length` is 10, which matches neither 7 nor 9 — so `Parse` returns null and the controller returns **400**. Correct, because an identifier is a fixed-width code, not a number; accepting leading zeros would mean `0009074729` and `9074729` are the same ship while `009074729` is a nine-digit MMSI.
2. Because leading zeros are significant and a `long` discards them. `"0009074729"` is the input that proves it.
3. `null`, hence a 400. The principle: a contradiction between explicit intent and the actual value is an error to report, not an ambiguity to resolve silently.

Chapter 19 — Controller

1. 400 means the request itself is malformed and will never succeed as written; 404 means the request was well-formed but the resource does not exist. Fix the request versus fix your expectations.
2. They imply different next actions. "Never heard of it" means the identifier is wrong or the ship is outside the bounding box. "Known but no position" means keep polling — it may resolve itself in seconds.

3. SQL lives in `AisRepository`, validation rules in `AisValidator`. The separation is what lets the controller be tested with mocks in milliseconds and the validator be tested with no infrastructure at all.

Chapter 20 — NavStatusMapper

1. Storage: a `TINYINT` is one byte against roughly 25 for the text, across millions of rows. Stability: if the wording changes you update one file rather than rewriting history — and translation stays possible.
2. It returns "Reserved" — wrong but not broken. Nothing crashes; the API simply reports a less useful label until you add the case.
3. Code 11 matches no arm, so the switch expression throws `SwitchExpressionException` — at **run time**, inside the controller, turning a display detail into a 500 response.

Chapter 21 — Schema

1. Standard SQL treats NULLs as equal for uniqueness, so a plain unique index would permit exactly one row with a NULL imo. The second vessel seen before its static message arrives would be rejected — which is most of them.
2. `vessels`: MMSI is globally allocated, unique, and never null, so it is a sound natural key. `positions`: no natural key exists, since a ship may legitimately report twice in the same second — hence a surrogate `IDENTITY`.
3. The key columns let SQL Server find the right rows in the right order; the `INCLUDE` columns carry the payload so the answer never touches the table. It serves `GetLatestPositionAsync`.

Chapter 22 — Tests

1. `[Theory]` with `[InlineData]`. Each data row is counted and reported as a separate test.
2. Removing `COALESCE` — every position report would wipe the vessel's name and IMO, and no unit test would notice. It needs an integration test against a real database: insert static data, then upsert from a position report, then assert the name survived.
3. It protects the deliberate decision to distinguish the two 404 cases — from a future developer who sees two nearly identical branches and "simplifies" them into one. The test fails and explains why the duplication is intentional.

WHERE TO GO NEXT

You now know this codebase better than most people know code they wrote themselves. Three things would extend it usefully, in increasing order of difficulty:

1. Fix findings 11 and 12 from Chapter 23 — two words and a string. Run the tests; watch them pass.
2. Fix finding 2 (the missing MMSI check on static messages) and add a unit test that would have caught it. This is the full loop: read, judge, fix, prove.
3. Add the `/health` endpoint the README lists as future work — reporting feed connection state and the age of the last message. It will force you to solve a real problem: the controller and the worker do not currently share any state, and Chapter 2 explains why that is deliberate. Deciding how to bridge that gap without ruining the separation is a genuine design exercise.